

Aprende a programar Python



Logo Python

Versión 0.99.2



Licencia CC by SA by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Septiembre 2022

Introducción

Historia de Python

Python fue creado a finales de los 80 por [Guido Van Rossum](#) (@gvanrossum en twitter) y desde entonces se ha ido mejorando y desarrollando por una enorme comunidad de usuarios. Si quieres saber más sobre su historia, puedes encontrar más detalles en la [página sobre la historia de Python en la wikipedia](#).

Algunas características de Python

- [Debe su nombre al grupo Monty Python](#), del que su creador Guido es un gran fan (no sé porqué he puesto este dato en primer lugar...).
- Python es open source, lo que quiere decir que se puede usar en todos los sistemas operativos y por supuesto totalmente gratuito. En [este repositorio de github, llamado cPython \(código C de Python\)](#) puedes encontrar su código fuente, al que han contribuido casi 1500 desarrolladores.
- En su [página web](#) podemos encontrar documentación y tutoriales de todos los niveles. También podemos descargar [desde su página](#) todas las versiones y para casi todos los sistemas operativos.
- La versión actual a día de hoy es la 3.10, estando en desarrollo la 3.11. Sobre la rama 3.8 y 3.9 se arreglan errores, sobre 3.7 y 3.6 sólo se arreglan temas de seguridad y la versión 2.7, presentada en 2010, se considera que ya llegó al fin de su vida útil en 2020.
- Existe la [Python Software Foundation \(PSF\)](#) que ayuda a la creación de eventos globales y locales. En 2019 entregó más de 300.000\$ en becas y ayudas en 60 países.
- ¿Para qué sirve Python? Es un lenguaje de propósito general, es decir podemos usarlo para hacer casi cualquier tipo de programa o aplicación. Por ejemplo:
 - Aplicaciones web: de hecho existen varios framework (entornos de desarrollo) como django, especializados en la creación de páginas web
 - Aplicaciones de escritorio en cualquier plataforma
 - Scripts (ficheros de lotes) para automatizar tareas repetitivas.
 - Creación de juegos, tenemos multitud de módulos que nos facilitan enormemente la creación de videojuegos.

- Es multiplataforma, es decir podemos usar nuestro código en cualquier plataforma (siempre que todos los módulos que usemos estén disponibles): Windows, MacOS, Linux, Unix,... De hecho viene instalado en los sistemas operativos Linux y MacOS, puesto que parte de los programas incluidos en estos lo usan.

Cuando decimos que es multiplataforma también nos referimos a que existe una versión de Python adaptada para ejecutarse en dispositivos embebidos y microcontroladores llamada [microPython](#) que funciona en equipos con reducidas prestaciones.

- Usa una sintaxis muy sencilla y fácil de comprender, lo que nos ayuda a aprender a programar leyendo otros programas. De hecho en su creación se puso un gran énfasis en conseguir un código muy legible.
- La estructura del código es más legible a simple vista.
- Es muy didáctico y uno de los lenguajes más adecuados para quien no ha programado nunca, y actualmente muchas universidades lo utilizan como lenguaje de entrada al mundo de la programación
- La curva de aprendizaje tiene una pendiente muy alta, pero un escalón inicial muy bajo, lo que nos permite empezar muy rápido a hacer cosas interesantes, casi desde el primer momento.
- Cuenta con multitud de librerías y módulos, lo que nos facilita mucho la tarea de programar, sólo tenemos que buscar si ya existe un módulo que nos sirva y adaptar algún ejemplo. Además estos módulos suelen ser también open source.

Algunos de estos módulos están programados completamente en Python y otros son recubrimientos de Python que internamente usan librerías escritas en C para optimizar el rendimiento

Podemos encontrar módulos para procesamiento de imágenes o para Data Mining (procesamiento masivo de datos) o para trabajar en IA (inteligencia artificial).

- Por todo esto se ha convertido en un estándar de hecho en el mundo de la ciencia, en todo lo relacionado con la investigación de IA, visión artificial, bioinformática, genética y en la investigación científica.
- Casi todas las grandes empresas del mundo del software utilizan Python, con Google a la cabeza, seguida por Facebook, Industrial Light & Magic (la de efectos especiales), Instagram, Spotify, Netflix, Dropbox,... hasta la mismísima NASA. En [esta página](#) puedes ver casos de éxito de desarrollos en Python.

- También son muchos los programas/aplicaciones/webs que usamos cada día y que no funcionarían sin Python, como por ejemplo Youtube, Google, Instagram, Reddit, Blender, Dropbox, BitTorrent, OpenShot,

Algunas características técnicas de Python

- Es interpretado: lo que quiere decir que nuestro código se va ejecutando línea a línea, sin que se haga una traducción global a un ejecutable. Eso nos permite que una de las formas de desarrollar nuestro código sea desde una consola interactiva, donde vamos añadiendo las órdenes y estas se ejecutan. Posteriormente podemos pasar estas órdenes a un archivo. Sin duda es una de las características más importantes de Python. También existen compiladores de Python que nos permiten convertir nuestro código a un ejecutable que el ordenador entiende directamente, pero no es la forma habitual de trabajar con Python

Al ser interpretado nos podemos encontrar con que el rendimiento de nuestros programas no es el mejor y no llegamos a exprimir la potencia del ordenador.

Con la potencia de los ordenadores de hoy en día y utilizando determinados módulos y características podemos mejorar mucho el rendimiento de nuestro código

Puedes encontrar algo más de detalle sobre este tema en el siguiente apéndice.

- Python no utiliza delimitadores para indicar los bloques de código como otros lenguajes ({...} en C, C++ o Java), si no que la indentación de las líneas (los espacios iniciales), marca estos bloques. Esto nos obliga a que la estructura del código sea más legible a simple vista.
- Tiene una comunidad muy activa, tanto en el desarrollo del lenguaje como en la documentación y la cantidad de módulos que se crean.

Apéndice: Compilados vs Interpretados

El compilador traduce nuestro código una única vez a lenguaje máquina que entiende el procesador. Este proceso puede ser lento, pero sólo se hace una vez.

El intérprete va traduciendo línea por línea nuestro código a un lenguaje intermedio que se ejecuta sobre unas librerías base, por lo no puede una optimización global.

Si estamos haciendo cambios continuamente sobre nuestro código, un lenguaje interpretado será más eficiente, pero una vez que tenemos un código definitivo, la compilación será mejor.

Tipo	Ventajas	Inconvenientes
Compilados	se hace una vez y se aprovecha muchas veces	Cada cambio implica una recompilación
Compilados	Ejecución más rápida	Hay que compilarlos en cada plataforma
Interpretados	Suelen ser de más alto nivel	
Interpretados	multiplataforma	

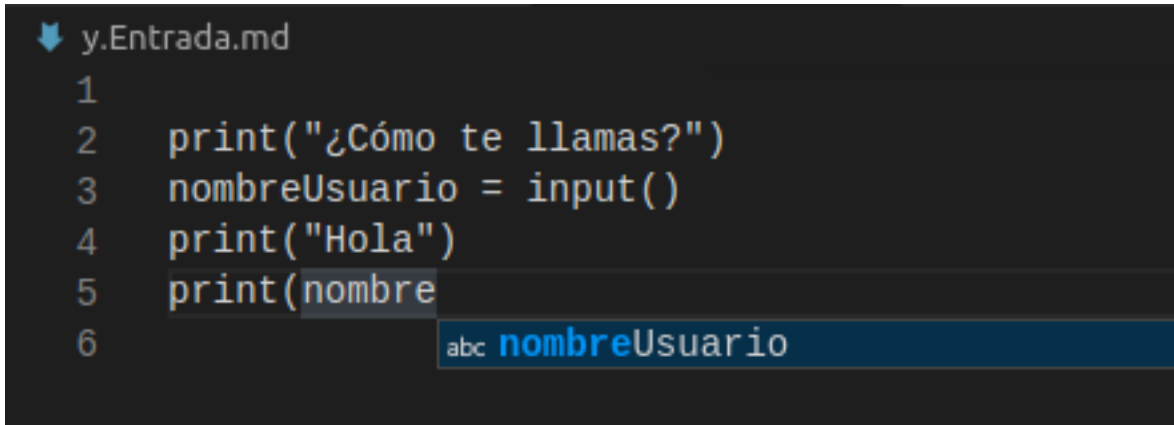
- Lenguajes Compilados: C, C++, Java
- Lenguajes Interpretados: Python, Perl, php, ruby

¿Cómo trabajar con python?

Ya que estamos convencidos de la utilidad de Python, vamos a comenzar con su instalación.

Tenemos 3 alternativas para trabajar con python:

- Instalar **sólo** el intérprete y usar cualquier editor de texto puro (no vale un Word o similar porque introducen caracteres de formato y el resultado no lo entendería el intérprete) para escribir nuestros programas, o
- Instalar un Entorno de desarrollo integrado (IDE) que nos facilite el trabajo. Para empezar a programar en serio debemos usar un IDE, que nos aporta ventajas y herramientas que nos facilitan el desarrollo como: correctores, escritura predictiva, depuradores, plantillas, etc.



```

y.Entrada.md
1
2 print("¿Cómo te llamas?")
3 nombreUsuario = input()
4 print("Hola")
5 print(nombre
6 abc nombreUsuario

```

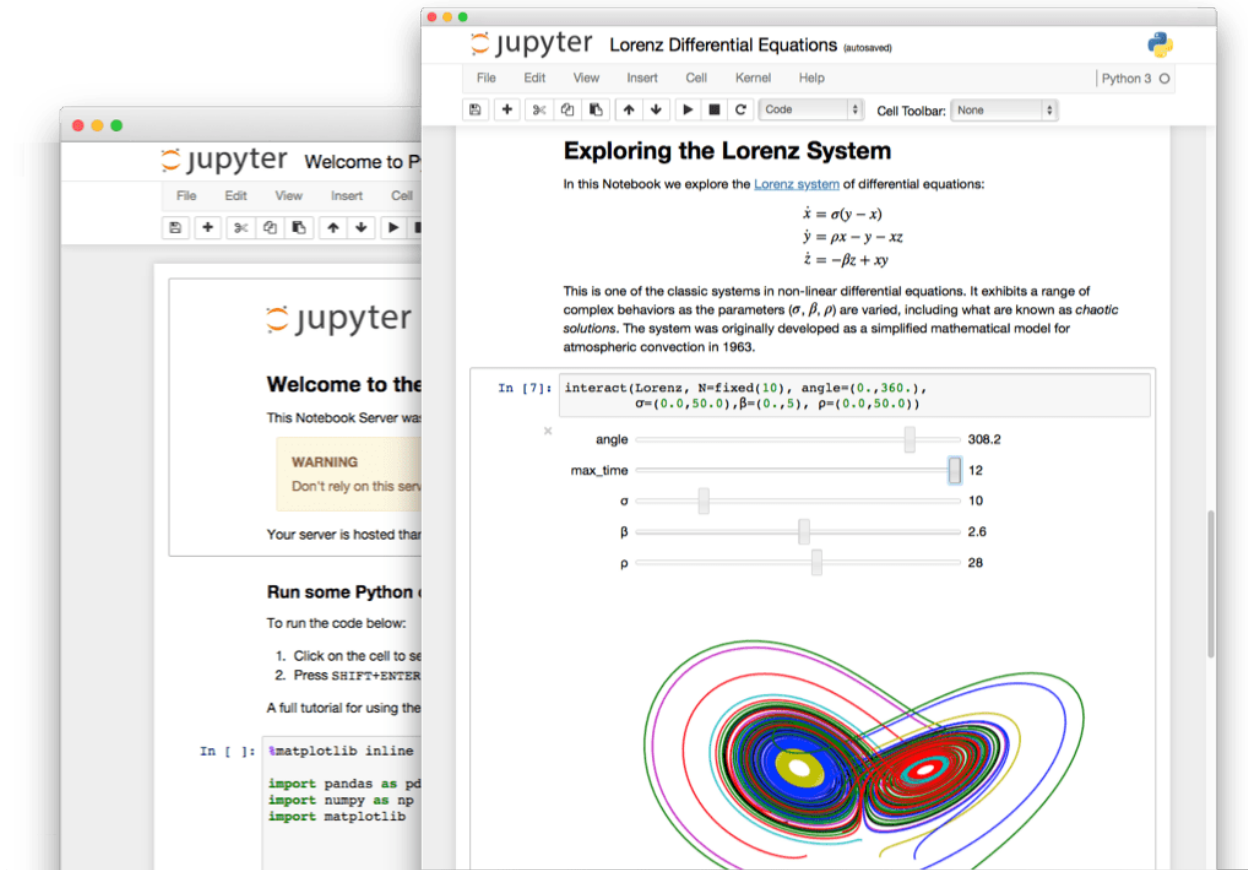
Ejemplo de escritura predictiva donde el IDE nos muestra las posibilidades para ahorrarnos tener que escribir toda la sentencia, ahorrando tiempo y evitando errores

- Utilizar un entorno de desarrollo web como [Jupyter Notebook](#) o [Google Colab](#) (que es la integración y evolución de Jupyter con Google Drive) que nos permite trabajar desde un navegador web. Nuestro código se ejecutará en un servidor, en nuestra máquina con Jupyter o en la nube de Google en el caso de Colab. En ambos casos tenemos integrado un gestor de ficheros y recursos y trabajamos con un documento donde se guarda nuestro documento y el texto/documentación de nuestro proyecto.

Para empezar a programar lo más sencillo es un entorno web como Colab o si planeas crear aplicaciones empezar con un entorno sencillo como Thonny que tiene todo lo necesario, pero que no nos abruma con un excesivo número de menús, ni de opciones que no necesitamos.

Jupyter Notebook

Jupyter es una aplicación Web (la ejecutamos en local, en nuestra máquina pero sobre un navegador web) diseñada para crear al mismo tiempo documentación y código pensando sobre todo en compartirlo posteriormente.



Jupyter preview

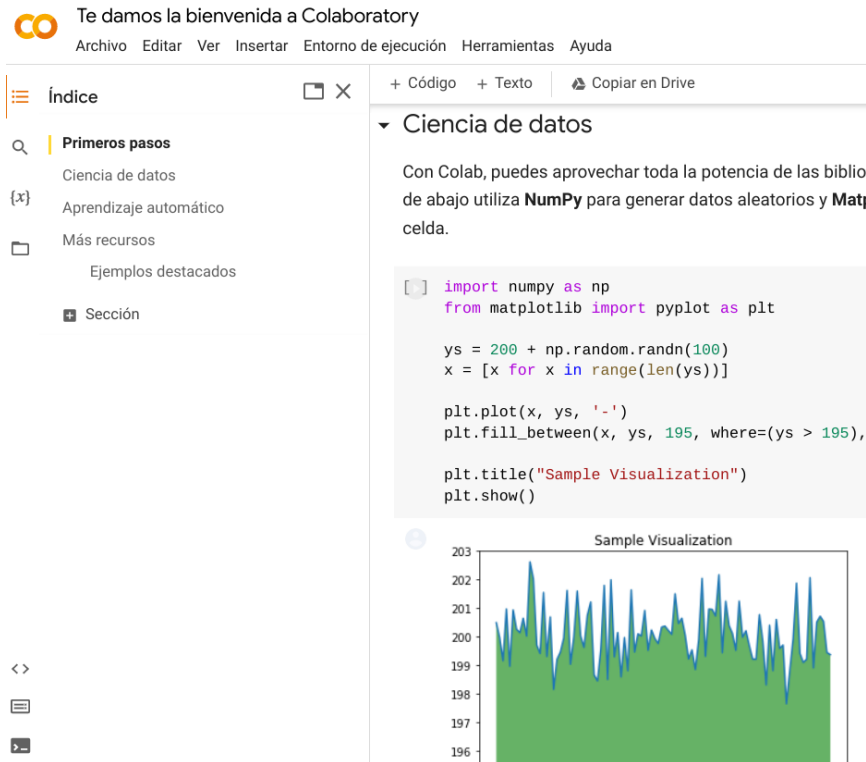
Como se puede ver en la imagen está muy orientada hacia el mundo científico sobre todo para el tema de gráficos, su entorno natural.

Google Colab

[Google Colab](#) es una adaptación de jupyter notebook por parte de Google y su integración con Google Drive, lo que nos permite compartir fácilmente con otros usuarios y también utilizar toda su infraestructura de la nube. Su nombre viene de Colaboratory lo que nos muestra claramente su vocación de trabajar colaborativamente.

Al igual que Jupyter Notebook, nos permite mezclar texto y código de manera que podemos trabajar en un artículo o en documento interactivo dónde podemos explicar, programar, calcular y al mismo tiempo presentar los resultados. También integra

herramientas de procesamiento y visualización de datos y en general data mining incluso de aprendizaje automático o machine learning.



The screenshot displays the Google Colaboratory web interface. At the top, a welcome message reads "Te damos la bienvenida a Colaboratory" with navigation links for Archivo, Editar, Ver, Insertar, Entorno de ejecución, Herramientas, and Ayuda. Below this is a sidebar with a search icon and a list of resources under "Índice": "Primeros pasos", "Ciencia de datos", "Aprendizaje automático", "Más recursos", "Ejemplos destacados", and "Sección". The main workspace is titled "Ciencia de datos" and contains a text block explaining that Colab allows using the power of libraries like NumPy and Matplotlib. Below the text is a code cell with the following Python code:

```
import numpy as np
from matplotlib import pyplot as plt

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195),
                color='green')

plt.title("Sample Visualization")
plt.show()
```

The output of the code cell is a line plot titled "Sample Visualization". The x-axis represents an index from 0 to 99, and the y-axis represents values ranging from 196 to 203. The plot shows a blue line representing the data points, which fluctuates around a mean of 200. The area under the line where the values are greater than 195 is filled with green, as specified in the code.

Otra gran ventaja de utilizar Google colab es que nos permite crear un entorno de ejecución totalmente personalizado para nuestro código Python, independiente de nuestro ordenador. Así podemos trabajar con los componentes o módulos que necesitemos instalados, incluso tenemos acceso a un entorno virtual donde se está ejecutando nuestro código.

Los cuadernos de Colab ejecutan código en los servidores en la nube de Google, lo que te permite aprovechar toda la potencia del hardware de Google, independientemente de la potencia de nuestro equipo

Un ejemplo típico de uso de Colab donde realmente aprovechamos muchas de sus posibilidades sería el compartir un documento donde ya tenemos preparado todo lo necesario para trabajar con un sistema de inteligencia artificial, con las librerías

preparadas, el código necesario para hacerlo incluso tenemos todas aquellas imágenes o objetos que queremos que nuestra inteligencia artificial aprenda.

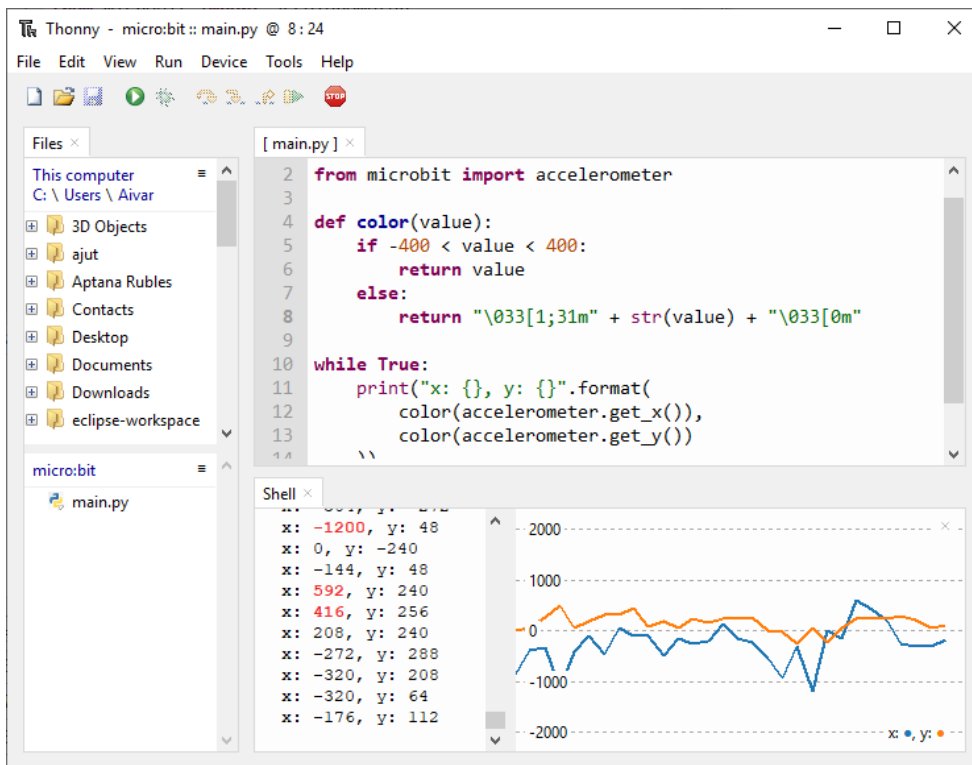
De esa forma, cuando compartimos nuestro documento con texto y con código, se está compartiendo la instalación, los ficheros, las librerías, imágenes, etc, en definitiva todo lo que necesitamos para trabajar, aprovechándose de la infraestructura de Google Drive.

Otra enorme ventaja es que estamos también delegando en los servidores de la nube de Google todo el procesamiento, nuestro ordenador no tiene que realizar aquí ese procesamiento sino que se hace todo directamente en los ordenadores de la nube de Google.

[Vídeo: presentación de Google Colab](#)

Thonny

[Thonny](#) es un IDE sencillo y está escrito en python! y es Open source. Puedes encontrar [su código fuente en github](#)



Thonny

El IDE Thonny incluye todo lo necesario para editar y ejecutar nuestros programas, puesto que además de editor incluye el entorno de ejecución de Python.

Por ello y por su facilidad de uso es la elección para quien empieza.

Thonny incluye las características esenciales para desarrollar en Python:

- **Edición de ficheros:** Podemos editar nuestros ficheros de código. También nos permite gestionar archivos con un explorador de ficheros.
- Como vemos **resalta la sintaxis**, lo que nos facilita la lectura.
- También nos ayuda **autocompletando código**, con variables y funciones. Esto nos agiliza la escritura del código y además evita errores.
- En la parte de abajo de la ventana vemos una **consola** que se conoce como **REPL** (Read-Eval-Print Loop) que nos permite trabajar interactivamente con Python
- **Depuración:** Podemos ejecutar nuestro programa de forma normal o en modo depuración donde podremos ejecutar línea a línea.
- **Visualización de variables y sus valores:** Mientras depuramos también podemos abrir la vista de variables que nos va a permitir seguir el valor que tienen las variables en cada momento.
- **Instalación de paquetes y complementos** Desde el propio IDE podemos gestionar e instalar paquetes y complementos, necesarios para nuestros programas.

José Antonio Vacas Martínez

Aprende a programar Python



Logo Python

Versión 0.99.2



Licencia CC by SA by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Octubre 2022

Primer programa: ¡Hola Python!

Vamos a empezar usando la consola REPL (a veces diré consola, otras REPL). Su propio nombre indica REPL (Read-Eval-Print Loop): es un bucle que lee nuestro código, lo evalúa y muestra el resultado continuamente. Cuando trabajamos así estamos usando el intérprete de Python en modo interactivo.

Nos va a permitir probar y ejecutar nuestro código de un modo interactivo. Yo lo uso para probar fragmentos de código que luego voy a introducir como parte de un programa.

Cuando nos referimos a un programa python, estamos hablando de un fichero, normalmente con extensión **.py**, donde hemos incluido órdenes para que se ejecuten de manera consecutiva. Cuando Python ejecuta uno de estos ficheros no funciona en modo interactivo, sino que ejecuta todas las órdenes del fichero hasta terminarlas todas.

Usando la consola

Todo lo que vamos a hacer en la consola de Thonny podríamos hacerlo en una consola de python cualquiera. La ventaja de Thonny es que ya nos da todo integrado y facilita enormemente el arrancar. En los sistemas que tienen Python integrado, como por ejemplo Linux, si escribimos en cualquier shell “python3”, accedemos a la REPL de Python.

Entramos en la consola de Thonny (recuerda comprobar que tenemos al menos una versión 3.9, si no es así entra ejecutando python3)

y ponemos tras el prompt “>>>”

```
print("¡Hola Python!")
```

Al pulsar la tecla Enter veremos que aparece en pantalla el texto

```
¡Hola Python!
```

A screenshot of a terminal window titled 'Consola' and 'Excepción'. The prompt is 'Python 3.8.0 (/usr/bin/python3.8)'. The user enters '>>> print(";Hola Python!")' and the output is ';Hola Python!'. The prompt '>>>' is followed by a vertical bar '|'.

```
Python 3.8.0 (/usr/bin/python3.8)
>>> print(";Hola Python!")
;Hola Python!
>>> |
```

"Hello Python" en la consola

En la imagen vemos que nos resalta la sintaxis, indicando con colores distintos cada parte del código.

También vemos en la imagen que aparece la versión del intérprete de Python que estamos usando.

También podemos hacer que se impriman números pero en este caso no son necesarias las comillas.

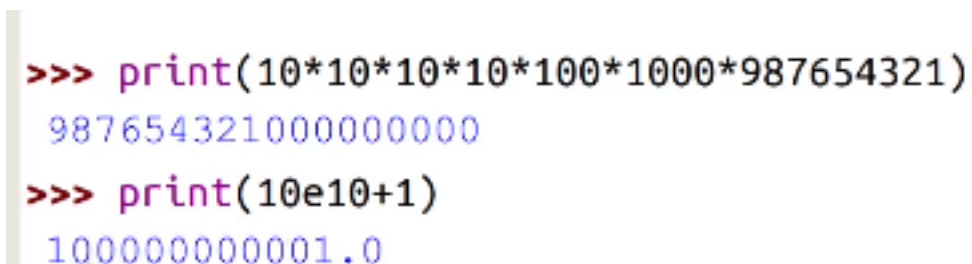
```
print(314)
print(3.14)
```

Podemos usar números enteros o decimales, usando el punto decimal "." como separador.

Incluso podemos poner operaciones ...

```
print(314 + 50)
```

Podemos hacer cálculos con muchas cifras y Python se porta realmente bien, incluso podemos usar notación científica:

A screenshot of a terminal window showing two Python commands and their outputs. The first command is '>>> print(10*10*10*10*100*1000*987654321)' and the output is '987654321000000000'. The second command is '>>> print(10e10+1)' and the output is '100000000001.0'.

```
>>> print(10*10*10*10*100*1000*987654321)
987654321000000000
>>> print(10e10+1)
100000000001.0
```

Cálculos matemáticos grandes

¿Qué ocurre si ponemos?

```
print("314 + 50")
```

La diferencia está en que al poner las comillas estamos diciéndole que muestre ese contenido literalmente, si no ponemos las comillas intenta evaluar la expresión.

Ejercicio Propuesto

1. Modifica este código para que imprima otro texto diferente

Uso de la Consola o REPL

Ya hemos trabajado un poco con la consola, y hemos visto que podemos ejecutar pequeños programas.

Vamos a resumir ahora algunas posibilidades que tiene:

- Podemos recuperar las últimas instrucciones usando las teclas “Flecha Arriba”  y “Flecha abajo”  del cursor del teclado
- Ctrl + L borra todo el contenido de la consola, pero seguimos pudiendo recuperar los comandos anteriores. Nos muestra el prompt de Python “>>>”
- Ctrl + D Reinicia la consola, reiniciando el intérprete. Nos mostrará la versión del intérprete que estamos usando.
- Ctrl + C Detiene el código que se esté ejecutando en ese momento.
- Podemos acceder a la documentación incluida en el intérprete de python con la función **help()**. Al ejecutar esta función entramos directamente en modo de ayuda interactivo y nos dará la documentación sobre las funciones cuyo nombre hemos indicado. Saldremos de la ayuda interactiva con “quit”
- También podemos usar la ayuda directamente dándole el nombre del que queremos obtenerla usando **help(función)**, como por ejemplo

```
help(print)
```

Errores de sintaxis

Vamos a ver con un poco más de detalle: Hemos usado la función (más adelante veremos qué es una función, pero me gusta ir adelantando términos para que se vayan fijando) **print()** con comillas simples ‘.’ el texto que queremos que aparezca. También podemos usar comillas dobles “.”. ¿Por qué esta variedad?, creo que por motivos históricos pero ahora nos permite hacer cosas como estas:

```
print('Hola "Python"')  
print("Hola 'Python'")
```

Siempre tenemos que tener cuidado de que las comillas funcionan como los paréntesis en matemáticas: El último en abrir el primero que tengo que cerrar. Por tanto no podemos alternar los tipos.

¿Y qué ocurre si no hacemos? Pues que se genera un error de sintaxis, también si nos olvidamos de poner alguna, o no cerramos los paréntesis....

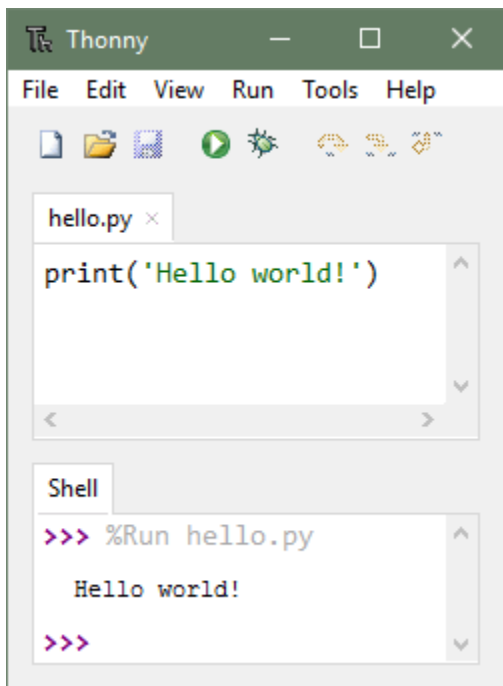
Afortunadamente Thonny nos ayuda resaltando nuestro código y si nos olvidamos de cerrar una cadena, nos lo indica marcando toda la línea en blanco con un color distinto. También ocurre algo similar si nos olvidamos de cerrar los paréntesis.

También se producen errores cuando ponemos alguna operación matemática incompleta.

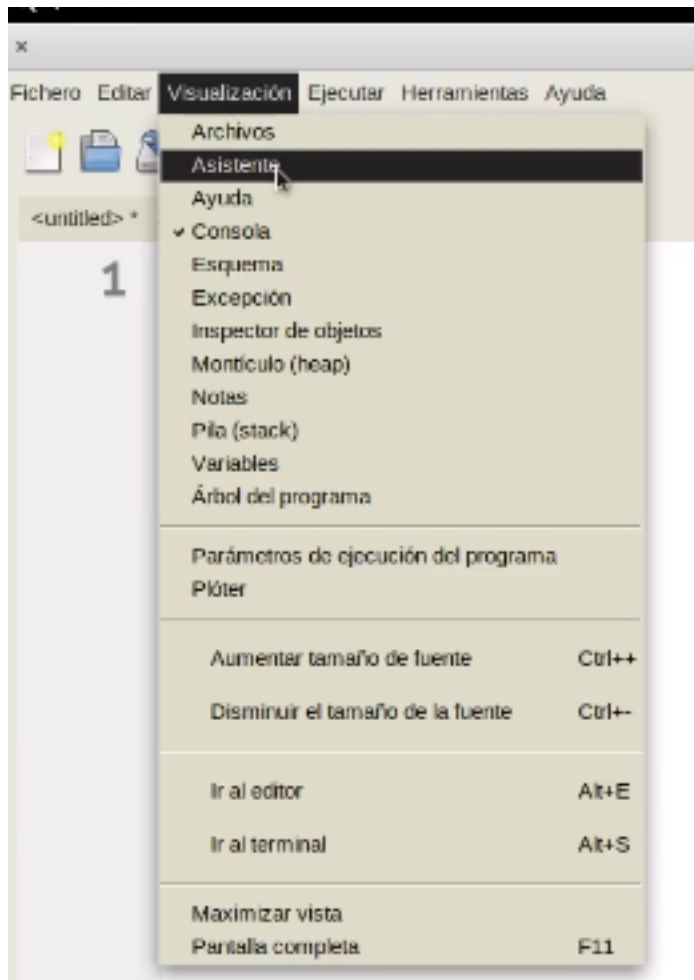
Uso del IDE Thonny

Vamos a repasar ahora algunas de las características de Thonny. Dejaremos las más avanzadas para cuando las necesitemos.

- La pantalla principal está dividida en la parte central para el editor de código y otras pestañas alrededor. En la parte de abajo está la consola:

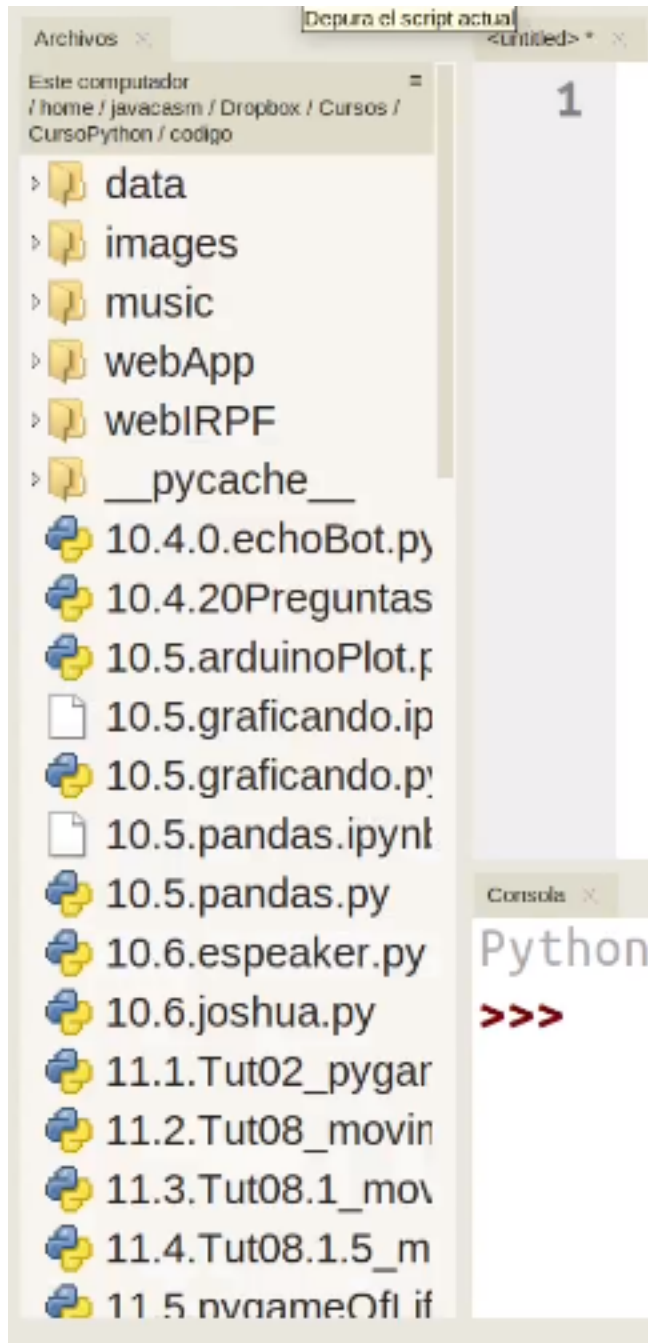


- Podemos abrir estas pestañas desde el menú **Visualización**



Aquí están las ventanas que más adelante nos permitirá ver los valores de las variables o la memoria.

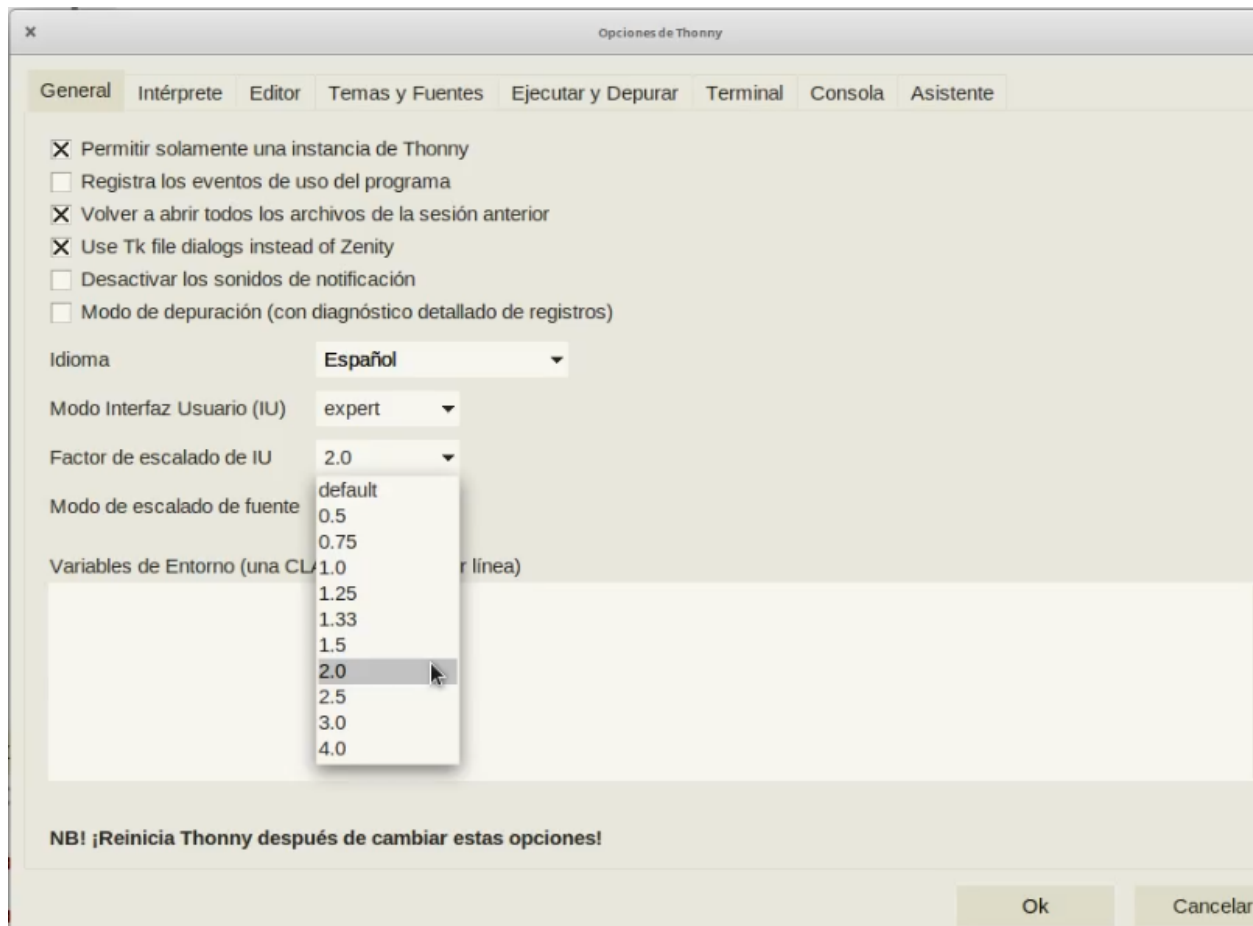
- Hay una pestaña que sirve como navegador de archivos, desde donde podemos navegar por carpetas, crearlas o borrar ficheros.



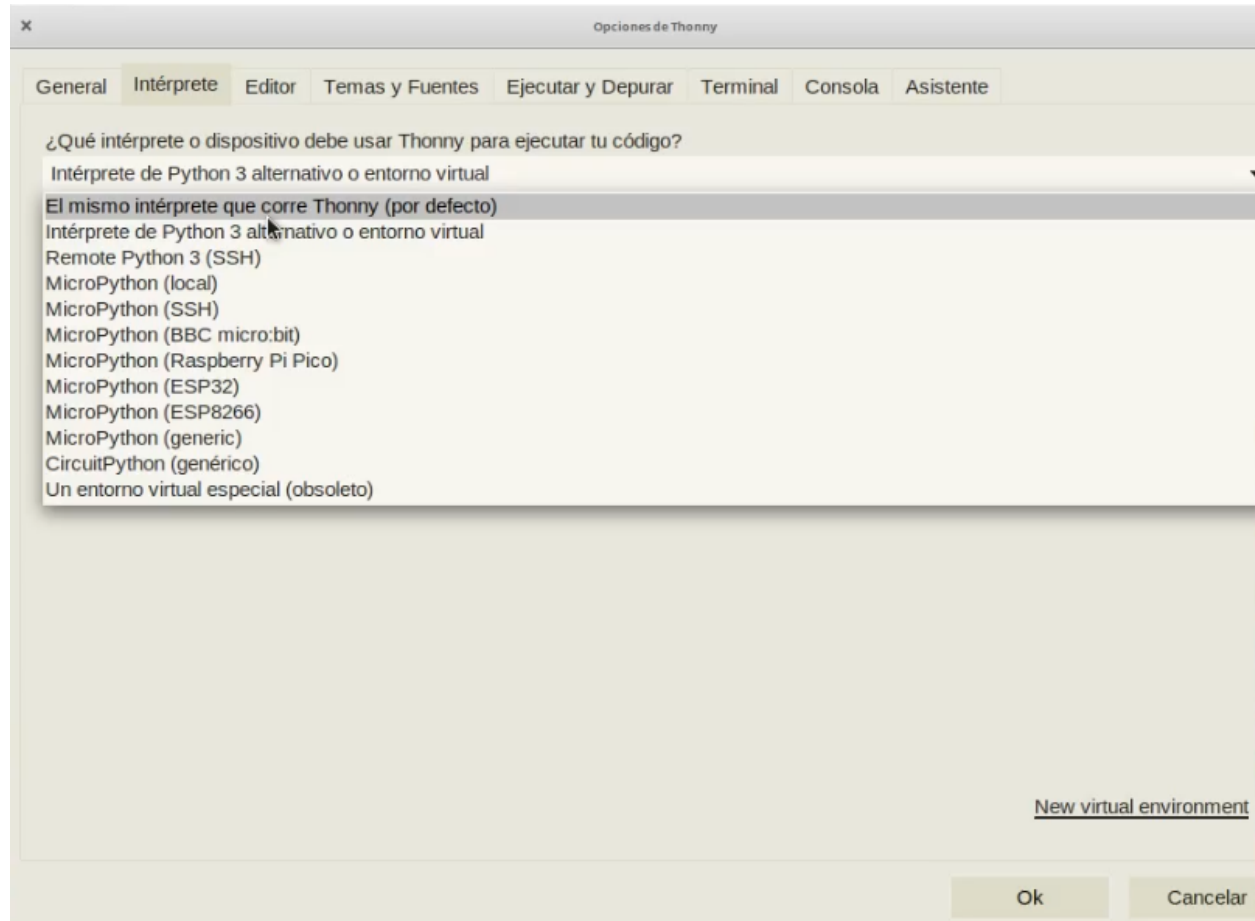
Pestaña navegador de Archivos

- Ctrl + "+" y Ctrl + "-" aumenta o disminuyen el tamaño del tipo de letra
- Desde la barra de herramientas podemos **Abrir**, **Guardar** y **Nuevo** los ficheros de código. Junto a ellos están los iconos de **Ejecutar** y **Depurar**, con los que trabajaremos en breve.

- En el menú **Edición** tenemos lo relacionado con **Copiar/Pegar** y algunas herramientas para el código que ya iremos viendo.
- En el menú **Herramientas** encontramos, las opciones para instalar paquetes Python y para instalar Complementos de Thonny
- También está el menú **Herramientas**, donde encontramos las **Opciones** que a su vez incluyen varias pestañas. Veamos algunas:
 - En la pestaña General podemos seleccionar el idioma y además yo le tengo puesto el **Factor de escalado UI** a 2.0 para que el interfaz se vea más grande.



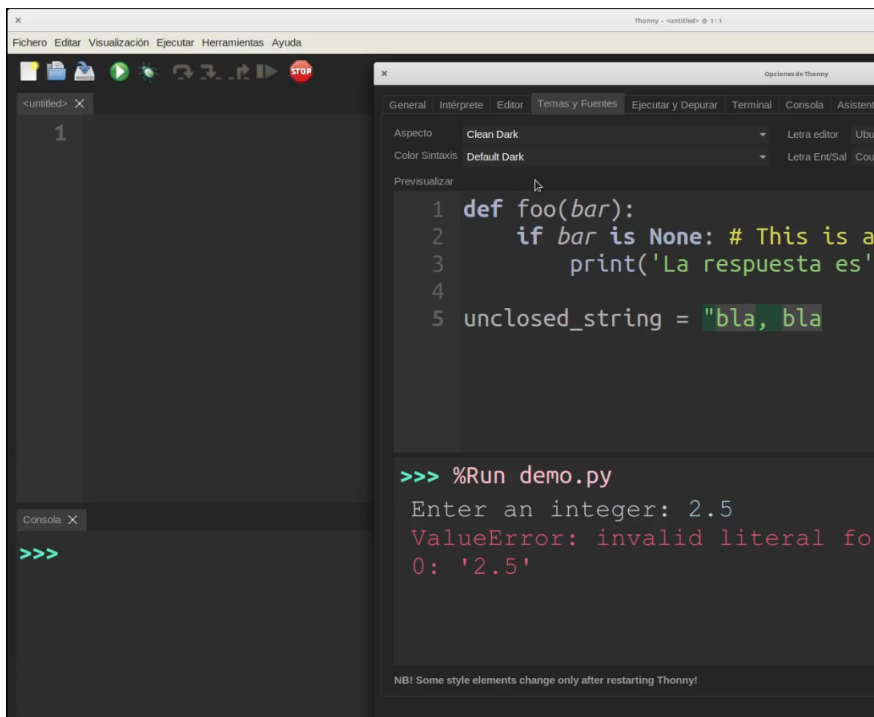
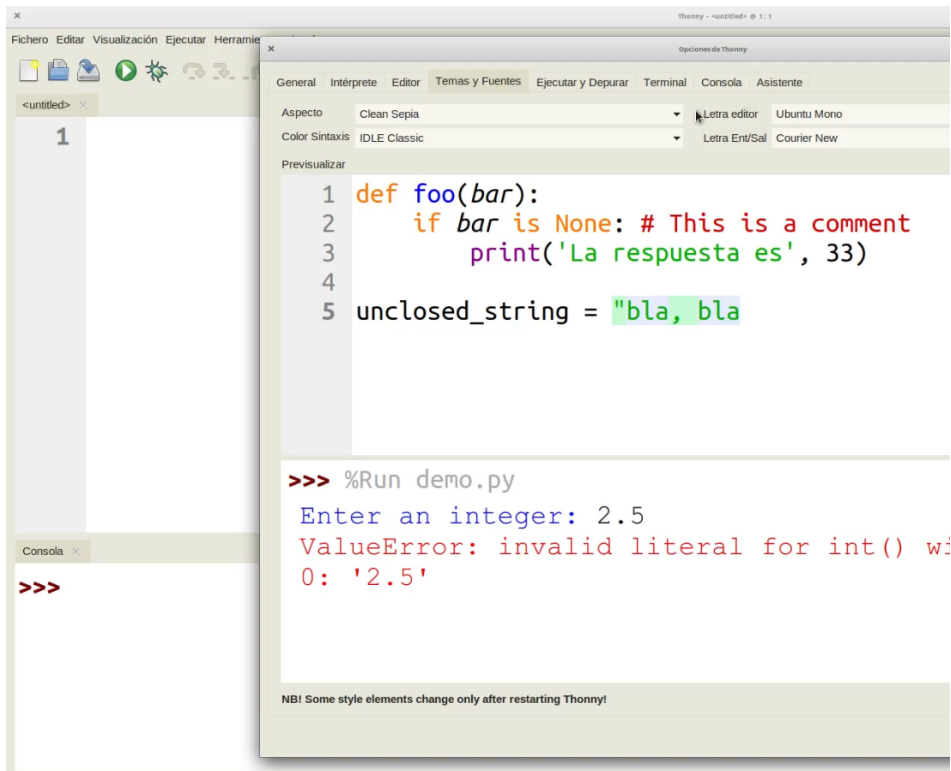
- En la pestaña **Intérprete** podemos seleccionar la versión de Python que usaremos. Normalmente se usa la “Incluída con Thonny” pero pudiera ocurrir que queramos usar otra



Thonny nos permite programar Python para todo tipo de equipos, es decir podemos programar en Python dispositivos diferentes a un PC, como son **micro:bit**, **ESP8266** o **ESP32**, en los que usaremos una versión reducida de Python, llamada Micropython, pero este lo veremos en otro curso.

- En la pestaña **Editor** configuramos cómo queremos que se resalte el código.
- En **Temas y fuentes** seleccionamos el aspecto, tema, tamaño de letra, etc. Cambiando el **Aspecto** y el **Editor** a Dark cambiaremos el aspecto de todo el entorno.

¿Cuál prefieres?



Usando Ficheros para guardar nuestro código

Desde la consola podemos ejecutar cualquier cosa pero las órdenes que usemos se perderán al salir

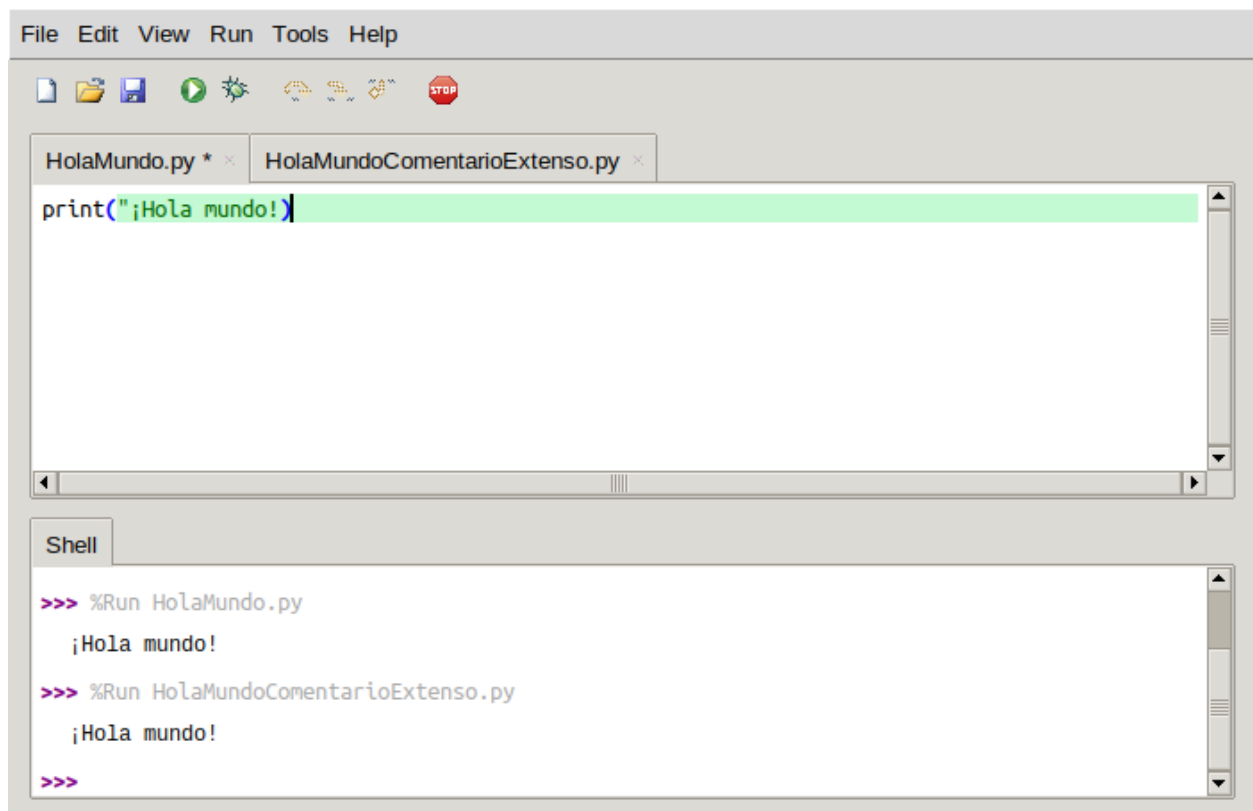
Por ello vamos a ver cómo podemos trabajar con ficheros donde incluiremos todas nuestras órdenes

Desde Thonny podemos crear carpetas y si somos ordenados crearemos una llamada “Hello World” para poner dentro el fichero correspondiente. Creamos un nuevo fichero que guardaremos en esa carpeta como **HolaMundo.py** con el siguiente contenido

```
print("¡Hola mundo!")
```

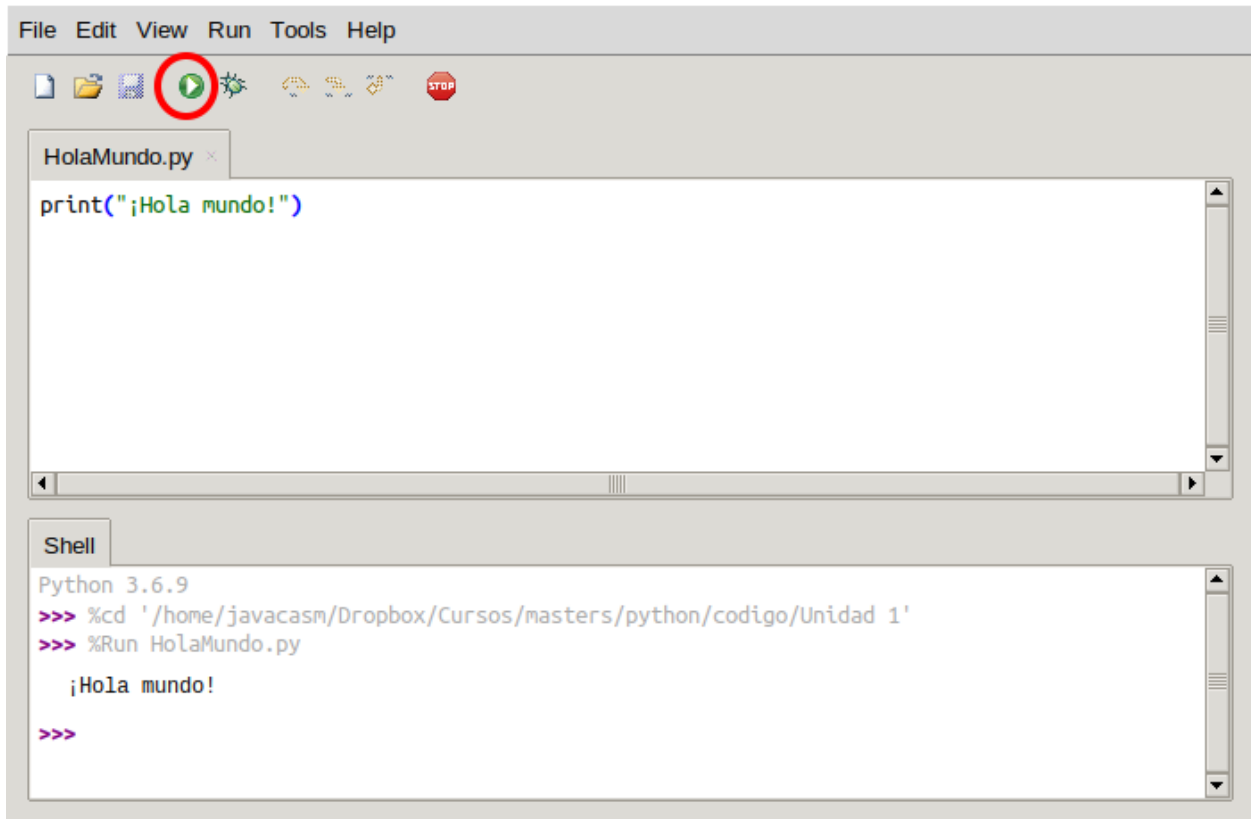
Se suele utilizar la extensión **.py** para indicar que el fichero contiene código python

Vemos que el editor resalta la sintaxis indicándonos las distintas partes. Esto nos puede ayudar si hemos cometido un error. Por ejemplo el editor Thonny nos resalta la línea si nos hemos olvidado de cerrar unas comillas



Error de sintaxis en Thonny

Desde el editor Thonny también podemos ejecutar nuestro programa pulsando el botón **Ejecutar**



```
# Nuestro primer programa en python que muestra el mensaje ¡Hola mundo!  
print("¡Hola mundo!")
```

Si queremos añadir un comentario que ocupe varias líneas podemos encerrar entre triples comillas dobles `"""` todo el texto

```
"""  
Nuestro primer programa en python que muestra el mensaje ¡Hola mundo!  
Escrito por @javacasm  
03/03/2021  
Licencia CC by sa  
"""  
print("¡Hola mundo!")
```

Si ejecutamos cualquier de las 3 versiones veremos que el resultado es exactamente el mismo

Cuidados al escribir código

Las líneas no pueden empezar con un espacio o tabulador porque es lo que usa Python para agrupar líneas, y ya veremos las normas que se tienen que cumplir

Detrás del signo de comentario de línea `#` da igual lo que pongamos, porque se ignora todo el resto de la línea

Depuración

Durante el proceso de creación de nuestro código, cuando estamos trabajando con nuestro programa nos encontramos con la necesidad de depurar el código. Es decir, de encontrar los posibles errores y revisar de una forma más o menos exhaustiva todos aquellos pasos que se van dando.

Para ello es muy cómodo el uso de las funciones de depuración que nos facilitan el ejecutar nuestro programa paso a paso, es decir línea a línea e ir viendo en cada uno de estos pasos los valores que van tomando las diferentes variables y cómo van funcionando las distintas partes de nuestro programa.



Iconos depuración

Veamos las distintas opciones de depuración y los iconos para usarlas



Depuración Thonny Play

Cuando ejecutamos normalmente nuestro programa, pulsando sobre el icono del “Play”, estas funciones de depuración no se pueden utilizar.



Depuración Thonny Debug

Para usarlas tenemos que ejecutar nuestro código en **Modo Depuración**, pulsando sobre icono del “bichito verde” (por si quieres saber el [origen del término bug](#))



Depuración Thonny Saltando

Cuando estamos ejecutando un programa en modo depuración podemos hacer que nuestro código se vaya ejecutando línea línea usando la opción **saltando**.

Veremos que la línea actual se resalta

```
1  """
2  Primer programa
3  Que he hecho con Thonny
4  """
5  print("Hola Python") # Primera linea print("hola")
6  print("Hola Mundo")  # segunda linea
```

Depuración Línea

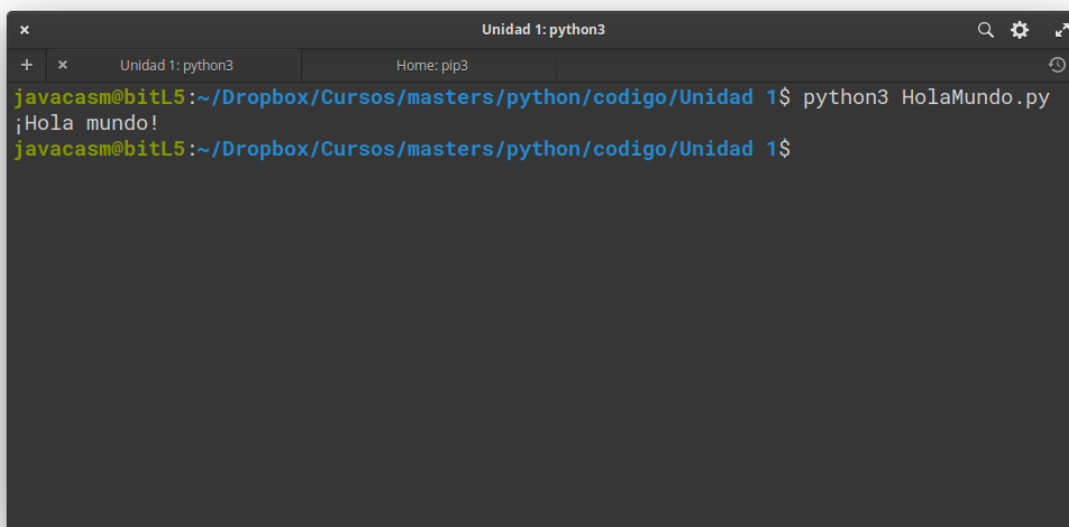
Ejecución desde línea de comandos

Si tenemos instalado el intérprete de comandos independiente o nuestro sistema lo incluye, podemos

```
python3 HolaMundo.py
```

(En algunos sistemas operativos el nombre de los ficheros es sensible a mayúsculas/minúsculas por lo que tendrás que ejecutarlo tal y como lo creaste)

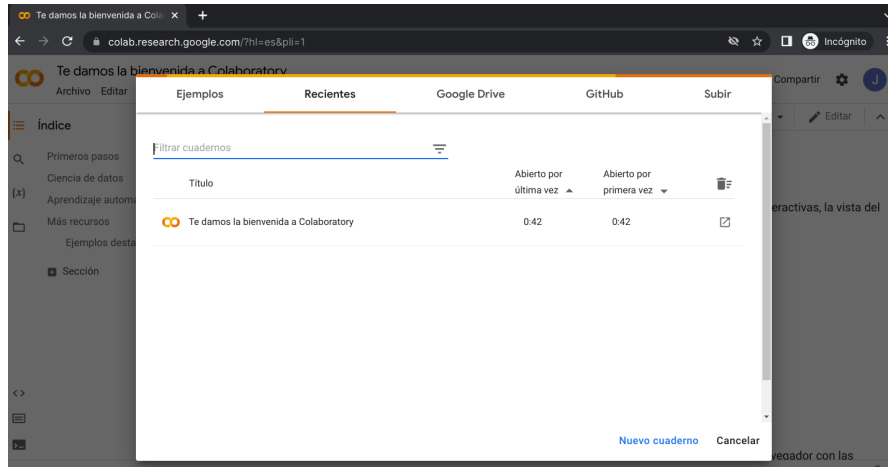
Y veremos el resultado

A screenshot of a terminal window titled 'Unidad 1: python3'. The terminal shows the command 'python3 HolaMundo.py' being executed, which outputs '¡Hola mundo!'. The prompt is 'javacasm@bitL5:~/Dropbox/Cursos/masters/python/codigo/Unidad 1\$'.

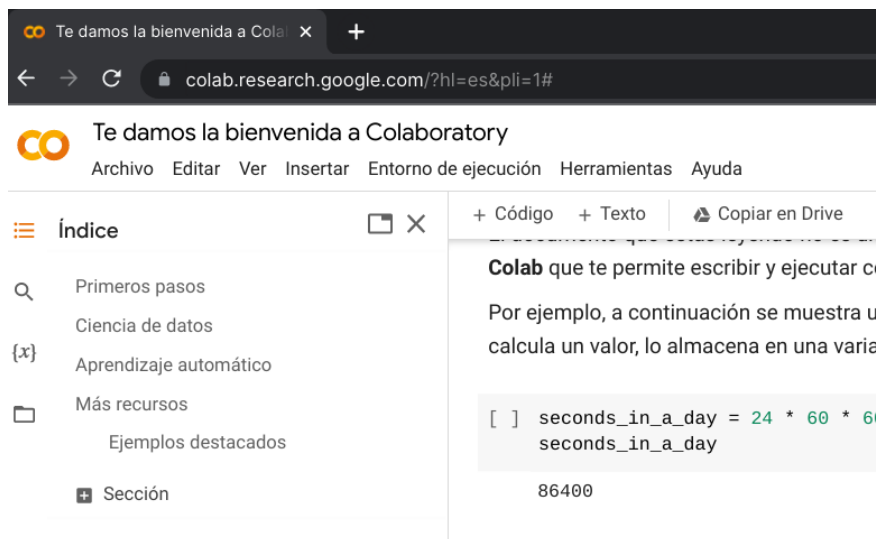
Empezando a usar Google Colab

Para empezar con Google Colab necesitamos una cuenta de gmail. Desgraciadamente a día de hoy los correos corporativos de Google Suite de EducaAnd no los incluyen entre sus aplicaciones.

Para empezar entraremos en la [página de Colab](#) e iniciamos sesión con nuestro usuario de correo de Gmail. Una vez dentro nos encontramos con un documento de bienvenida:



En este documento tenemos un tutorial, con vídeos, para aprender a usar Google Colab.



Primer programa: ¡Hola Python!

Es una costumbre de todo programador novato, que el primer programa ha de mostrar en pantalla un saludo.

Para crear nuestro primer documento Colab:

- Pulsamos en el botón “Nuevo Cuaderno” o desde el menú “Archivo” -> “Nuevo Cuaderno” y veremos un nuevo documento que recuerda bastante a los de Google Suite.
- Pulsamos sobre el nombre del documento “Untitled0” y lo renombramos por “Hola Mundo”

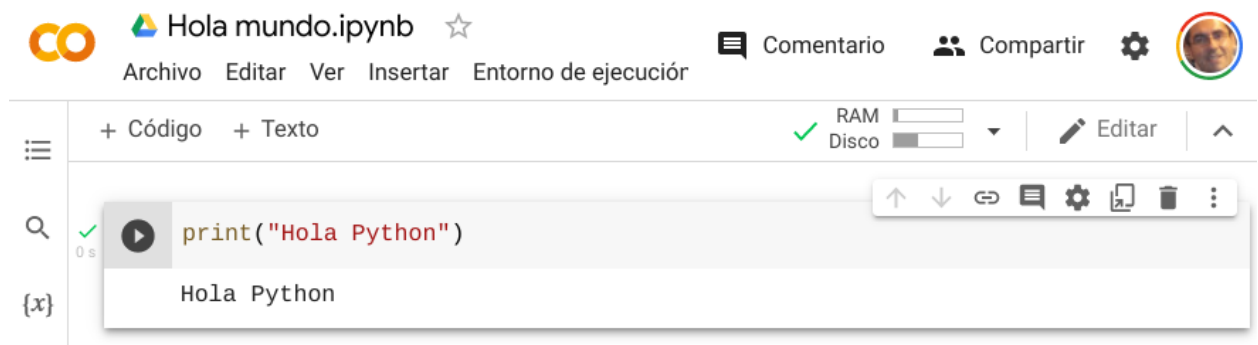
- Escribimos nuestra primera línea de código:

```
print("Hola Python")
```

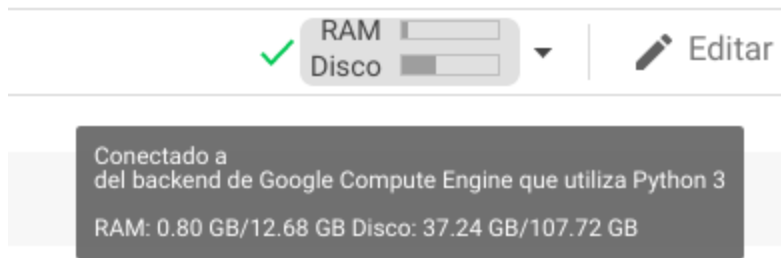
que significa: Muestra en pantalla el texto entre comillas (podemos usar comillas simples `'...'` o comillas dobles `"..."` pero en cada texto del mismo tipo)



- Ahora pulsamos el círculo con el signo Play y nuestro código se ejecutará, mostrando el texto que había entre comillas. También podemos ejecutar el código pulsando Ctrl+Enter.



Cuando ejecutamos por primera vez vemos que se muestra un mensaje de “Conectando con el servidor” y tras la conexión aparecen los recursos que estamos consumiendo, en forma de memoria RAM y de almacenamiento en Disco:



Si dejamos el ratón encima nos muestra los límites que tenemos con nuestro usuario actual. Si necesitáramos más podemos contratarlo.

Si queremos cambiar el texto podemos tocar sobre el código y modificar la parte entre comillas.

Podemos añadir otra línea de código pulsando sobre “+ Código” y añadir código para mostrar distintos textos.

También podemos añadir textos que se mostrarán en nuestro documento. Podemos conseguir formatear el texto usando los botones del bloque de texto o un sencillo formato llamado Markdown.

Mezclando texto y código podemos conseguir generar documentos interactivos de manera muy sencilla.

En la imagen vemos que en nuestro código nos resalta la sintaxis, indicando con colores distintos cada parte del código.

También podemos hacer que se impriman números, en este caso no son necesarias las comillas y usaremos

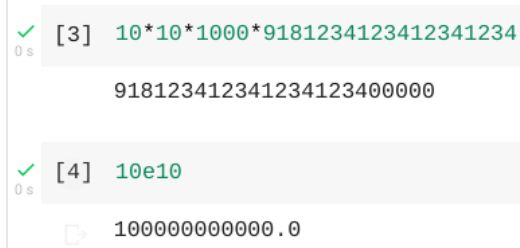
```
print(314)
print(3.14)
```

Podemos usar números enteros o decimales, usando el punto decimal “.” como separador.

Incluso podemos poner operaciones usando los tradicionales operadores aritméticos (usando el asterisco “*” para indicar multiplicación)

```
print(314 + 50)
```

Podemos hacer cálculos con muchas cifras y Python se porta realmente bien, incluso podemos usar notación científica (10e6 para 1000000):



```
[3] 10*10*1000*9181234123412341234
918123412341234123400000

[4] 10e10
100000000000.0
```

¿Qué ocurre si ponemos?

```
print("314 + 50")
```

La diferencia está en que al poner las comillas estamos diciéndole que muestre ese contenido literalmente, si no ponemos las comillas intenta evaluar la expresión.

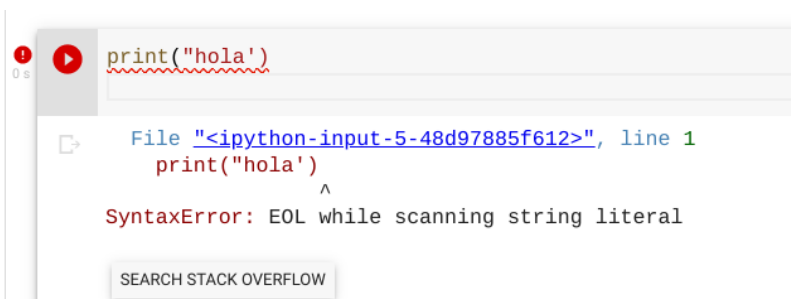
Errores de sintaxis

Vamos a ver con un poco más de detalle: Hemos usado la función (más adelante veremos qué es una función, pero me gusta ir adelantando términos para que se vayan fijando) **print()** con comillas simples `'..'` el texto que queremos que aparezca. También podemos usar comillas dobles `".."`. ¿Por qué esta variedad?, creo que por motivos históricos pero ahora nos permite hacer cosas como estas:

```
print('Hola "Python"')
print("Hola 'Python'")
```

Siempre tenemos que tener cuidado de que las comillas funcionan como los paréntesis en matemáticas: El último en abrir el primero que tengo que cerrar. Por tanto no podemos alternar los tipos.

¿Y qué ocurre si no hacemos? Pues que se genera un error de sintaxis, también si nos olvidamos de poner alguna, o no cerramos los paréntesis....



```
print("hola')
```

File "<ipython-input-5-48d97885f612>", line 1
print("hola')
^
SyntaxError: EOL while scanning string literal

SEARCH STACK OVERFLOW

Afortunadamente Colab nos ayuda resaltando nuestro código y si nos olvidamos de cerrar una cadena, nos lo indica marcando toda la línea en rojo. También se producen errores cuando ponemos alguna operación matemática incompleta.

Uso del editor de Colab

- Ya hemos visto que podemos editar cualquier línea escrita tanto de código como de texto o de su formato
- Podemos pulsar entre las bloques ya escritos y añadir contenidos de cualquier tipo
- Igual que con cualquier documento de Google Suite, podemos copiarlo o compartirlo con otros usuarios, o ver el historial de cambios.
- También podemos descargar desde “Archivo” -> “Descargar”

Aprende a programar Python



Logo Python

Versión 0.99.2



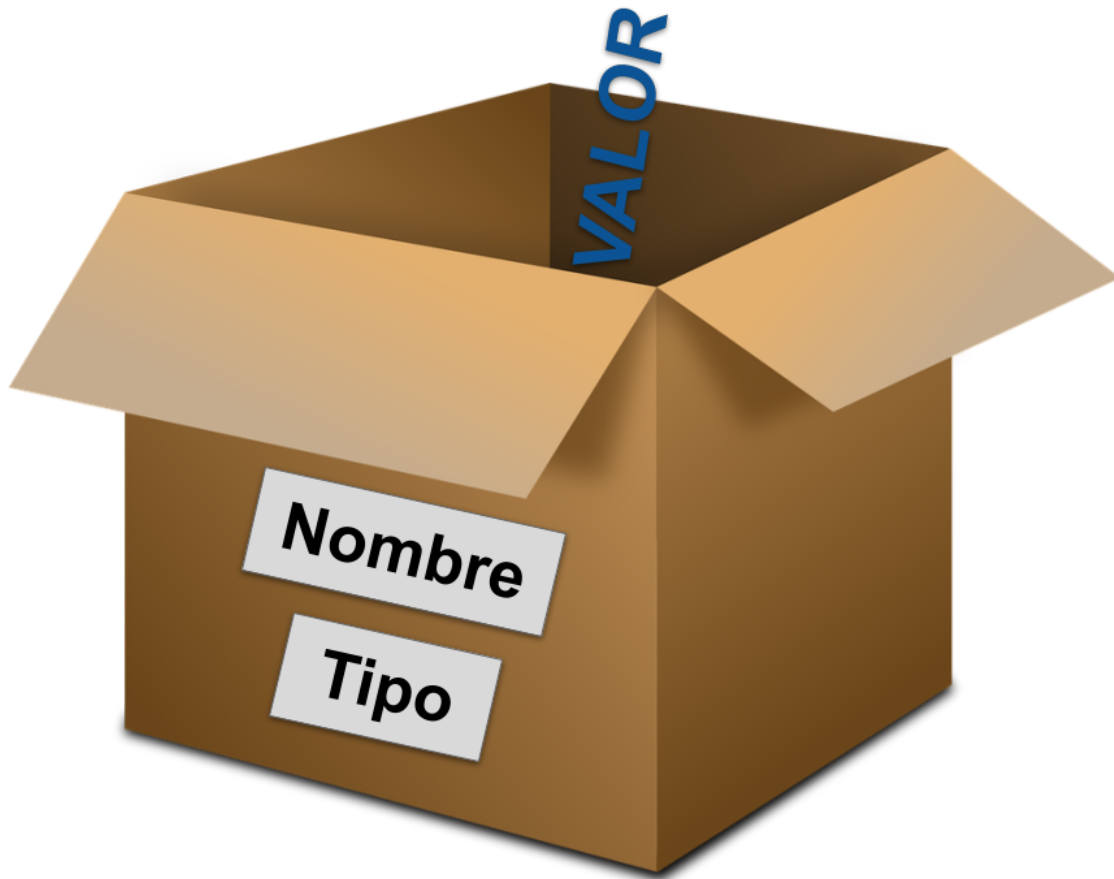
Licencia CC by SA by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Octubre 2022

Variables



Variable

En este tema vamos a empezar a trabajar con variables, que son la forma que tenemos de almacenar datos en nuestro programa.

Podemos imaginar una variable como una caja a la que le damos un nombre y que es capaz de almacenar un valor, que puede cambiar. Las variables tienen también un tipo, que en Python viene definido por el valor que le damos.

Aunque no tenemos que decirlo explícitamente en Python las variables tienen tipo

En la siguiente línea estamos creando un variable y le damos el valor **Pepe**

```
persona = "Pepe"  
print("Hola")  
print(persona)
```

Llamamos asignación al hecho de dar a una variable un valor dado.

Diremos que hemos declarado la variable **persona** y le asignamos su valor.

Posteriormente podemos cambiar su valor sin más que asignarle otro nuevo.

Vemos cómo distinguimos cuando queremos imprimir un texto literal y el valor de la variable

Para indicar el valor de las variables de tipo texto podemos comillas simples ' o dobles "

Los nombres de las variables pueden estar formados por letras (mayúsculas y minúsculas, entre las que se distingue), números y el signo "_" de guión bajo, pero no pueden contener espacios, ni empezar con números. Se recomienda que no se usen letras con acentos o ñ, si bien todo esto depende más del editor que usemos y cómo codifique el fichero.

Se recomienda usar nombres claros y descriptivos. Podemos usar guión bajo o separarlas por las letras mayúsculas/minúsculas (a esta técnica se la llama [camelCase](#)). Por ejemplo:

```
nombrePersona = "Pepe"
```

ó

```
nombre_persona = "Pepe"
```

De la misma forma, aunque no es obligatorio se suele espaciar las asignaciones y las operaciones para hacerlos más legibles

Palabras reservadas

Existen ciertas palabras que tienen otro uso en python y no podemos usarlas como nombres de variables. Puedes encontrar el listado [aquí](#) para la versión 3.7 de python

Palabras	reservadas	en python		
False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Podemos ver la lista de las palabras reservadas para nuestra versión de Python usando **help('keywords')**

Entrada

Ya que tenemos dónde guardar datos vamos a ver cómo podemos pedir datos al usuario usando el teclado para ello.

La función que nos permite hacer se llama **input** (poca imaginación con el nombre...) y podemos indicar que queremos que se muestre un texto cuando se pide el dato.

```
nombreUsuario = input("¿Cómo te llamas? ")
print("Hola " + nombreUsuario)
```

Recuerda dejar un cierto espacio entre el final de la pregunta y nuestra respuesta. Podemos usar un espacio, o hacer que se salte a la línea siguiente con *******

La función **input** siempre nos va dar un valor de tipo **Cadena**, independientemente de lo que introduzca el usuario, que deberemos convertir al tipo de datos que necesitamos

```
nombreUsuario = input("¿Cómo te llamas?\n")
print("Hola " + nombreUsuario)
```

Tipos

Cuando asignamos un valor a una variable le estamos dando un tipo a la variable. Los tipos pueden ser numéricos, que podrán ser **enteros** o **decimales**, de tipo **cadena**, o **booleanos** (de valor lógico) que sólo pueden ser **True** o **False**

En Python las variables tienen un tipo que se infiere a partir del contexto en la asignación es decir no es necesario explícitamente decírselo,

Para expresar números decimales usaremos el **.** como separador decimal

```
# variable decimal o float
pi = 3.1416
```

También podemos usar variables lógicas o booleanas que podrán ser **True** o **False** (hay que escribirlas así)

```
# variable booleana o lógica
respuesta = True
```

Cuando asignamos un texto usamos las comillas, pero no para los números

```
edad = 10
nombre = "Juan"
print(nombre)
```

```
print(" tiene ")  
print(edad)
```

Dado que muchas veces necesitamos imprimir varias cosas de manera consecutiva, podemos separar los valores por comas. Así el ejemplo anterior quedaría así:

```
edad = 10  
nombre = "Juan"  
print(nombre, " tiene ", edad)
```

Conviene destacar que es diferente hacer

```
a = 10
```

que

```
a = "10"
```

Con el primero podremos hacer operaciones aritméticas

```
a = 10  
b = 15  
c = a + b  
print(c)
```

Obtenemos el esperado resultado de **25**

Pero si usamos cadenas el resultado puede no ser el esperado

```
a = "10"  
b = "15"  
c = a + b  
print(c)
```

Obtenemos **1015** que es el resultado de concatenar las dos cadenas

También podemos hacer todas estas operaciones en la consola de python

```
>>> a = 10
```

Si necesitamos crear una cadena que incluya finales de línea (") podemos delimitarlas con "" (como los comentarios extensos)

```
>>> texto = '''En un lugar de la Mancha  
De cuyo nombre no quiero acordarme...'''  
>>> print(texto)
```

Dentro de una cadena podemos incluir algunos caracteres especiales como son:

- '\n' final de línea
- '\t' Tabulador

- `\r` Vuelve a principio de línea, pero sin pasar a la siguiente línea
- `\\` El carácter
- `\'` La comilla simple
- `\''` La comilla doble

Si en algún caso tuviéramos que incluir estos caracteres, pero no queremos que tenga ese sentido podemos evitarlo añadiendo una letra `'r'` antes de la primera comilla.

```
print(r'a\na') # se imprime en 1 sola línea
```

Como hemos visto también existen las variables de tipo booleano, que sólo pueden contener 2 valores **True** o **False**, que han de ser escritas de esta manera.

```
isOpen = True
```

```
isVisible = False
```

Asignaciones múltiples

Podemos asignar el valor a varias variables en una misma línea sin más que separar los nombres a un lado del igual por comas y también los valores al otro lado

```
edad, nombre = 10, "Juan"
```

```
valor1 = valor2 = valor3 = 6
```

Comprobación de tipos

Podemos comprobar el tipo de cualquier variable usando la función **type()**.

Así:

```
>>> text = 'hola'
>>> precio = 3.14
>>> posicion = 100
>>> print(type(precio))
<class 'float'>
>>> print(type(text))
<class 'str'>
>>> print(type(posicion))
<class 'int'>
```

Podemos comprobar si es de un tipo u otro usando la función **isinstance(variable, tipo)**, por ejemplo así:

```
>>> print(isinstance(posicion,int))
True
>>> print(isinstance(posicion,float))
False
>>> print(isinstance(precio,float))
True
>>> print(isinstance(text,str))
True
```

Tipos de variables

A veces es necesario convertir el valor de una variable a otro tipo. Veamos un ejemplo

```
nombreUsuario = input("¿Cómo te llamas?")
print("Hola ",nombreUsuario)
fechaNacimiento = input("¿En que año naciste?")
edad = 2020 - fechaNacimiento
print("Tu edad es ",edad)
```

Al ejecutarlo encontramos un error, que viene de que la función **input** siempre guarda el valor como una cadena y python no sabe cómo usar un número con una cadena. Por ello lo que tenemos que hacer es guardar el valor del año de nacimiento como un entero. Para ello usaremos la función **int()** que permite convertir el valor de una variable a un tipo entero

```
nombreUsuario = input("¿Cómo te llamas? ")
print("Hola ",nombreUsuario)
fechaNacimiento = int(input("¿En que año naciste? "))
edad = 2020 - fechaNacimiento
print("Tu edad es ",edad)
```

Existen otras funciones de conversión entre tipos

Conversión	Función
int()	a entero
float()	a decimal
str()	a cadena
bool()	a booleano

Comprobación de tipos numéricos

Podemos comprobar si el valor de una variable numérica es entero o no. Para ello usaremos la función **is_integer()** de la siguiente forma:

```
x = 1.23
x.is_integer()
```

```
y= 1.0  
y.is_integer()
```

Excepciones

A veces nos podemos encontrar con errores de ejecución en nuestro código y debemos de estar prevenidos. Pueden ser debidos a errores del código y como programadores los iremos corrigiendo, mejorando así la calidad del código.

También hay errores que se pueden producir por circunstancias externas, como por ejemplo cuando no podemos acceder a un archivo, o cuando hay un problema de acceso a la red que no está disponible. Son lo que se conoce como **errores de tiempo de ejecución (Runtime errors** en inglés) y un buen programa siempre tiene que estar protegido contra ellos.

También es necesario proteger nuestros programas contra el mal uso de los usuarios, que pueden ser mal intencionados o por despiste.

No hay que confundir estos errores con los errores de sintaxis por código mal formado. Existe un mecanismo para tratar estos errores en tiempo de ejecución, cuando nuestro programa se está ejecutando.

Cuando se produce un error de estos, decimos que se genera una **excepción**, y cuando se produce una nuestro programa se detiene en esa línea y se para.

Podemos hacer que nuestro código capture estas excepciones y así evitar que nuestro programa finalice. Para ello usaremos la estructura **try: ... except :**, que se divide en 2 partes, a las que llamaremos bloques:

- Ponemos **try:** antes de la parte inicial donde pensamos que se puede producir la excepción.
- Ponemos **except:** para indicar el código que queremos que se ejecute si se produce la excepción.

Para delimitar claramente el código de cada parte añadimos unos espacios (normalmente 4) antes de las líneas de código de cada bloque.

Python no utiliza delimitadores para indicar los bloques de código como otros lenguajes (**{...}** en C, C++ o Java), si no que la indentación de las líneas (los espacios iniciales), marca estos bloques.

```
# Comprobar si una cadena es o no un número  
s = input('Introduzca un número entero: ')
```

```
try: # Comienzo del bloque try
    valorInt = int(s) # Se produce una excepción
    print('Es entero') # Esta línea no se ejecuta
except: # Comienzo del bloque except
    print('No es entero')

print('Hemos terminado') # Esta línea no está dentro de los bloques
```

Existen diferentes tipos de excepciones, según el error que se produzca. Las que se generan por conversión decimos que son de tipo **ValueError**

También podemos lanzar una excepción si se ha producido un error ¿Por qué lanzar una excepción en lugar de hacer algo para corregirlo? porque quizás no está en nuestra mano el evitar el error.

Imagina que hemos hecho un código que calcula el precio de la cuota de una hipoteca y que nos piden que lo hagamos para un importe con valor negativo, para el que claramente no tiene sentido. Si no podemos volver a pedir que nos proporcionen un valor correcto podemos lanzar una excepción indicando el error en los valores proporcionados.

Para hacerlo usaremos la palabra reservada **raise** seguida de **Exception** a la que le podemos pasar un texto explicativo. Por ejemplo, si queremos hacerlo en el caso anterior del cálculo para un capital negativo haríamos:

```
## Comprobamos que el valor no es válido
raise Exception('El capital debe ser mayor que 0')
```

Cuando lanzamos una excepción, en alguna parte del programa se tendrá que capturar (con un try/except) o de lo contrario el programa se detendrá.

Operadores Aritméticos

Entre los valores numéricos podemos hacer operaciones matemáticas. Estos son los operadores aritméticos ya definidos

Operador	Operación
+	suma
-	resta
/	división (con decimales)
//	división entera
%	módulo o resto de una división

****** potencia

Podemos hacer operaciones entre número y entre variables usando estos operadores.

Algo muy frecuente es que queramos incrementar o decrementar el valor de una variable para ello podemos hacer una versión simplificada poniendo detrás del operador el signo “=”. Así: “a = a + 10” se puede simplificar a “a += 10”, pudiendo hacerse con todos los operadores

```
a = 10
a += 5
a *= 10
a /= 7
```

Python no tiene límite en cuanto al tamaño máximo que puede usar para números enteros

```
a = 10 ** 1000
print(a)
```

Cuando operamos con variables de distintos tipos pero que sean numéricos (bool, int y float) se producen conversiones pasando al tipo más complejo, según la siguiente tabla

Tipo 1	Tipo 2	Resultado
bool	int	int
bool	float	float
int	float	float

Ahora ya podemos empezar a hacer una calculadora que sume 2 números, pero ya vimos que tenemos que darnos cuenta de que las cadenas se suman de forma diferente...

```
a = "10"
b = "15"
c = a + b
print(c)
```

Decimos que se concatenan. Para sumarlos correctamente los convertiremos en enteros.

```
# Programa que realiza la suma de dos valores
a = input('numero 1 ') # pedimos el primer número
b = input('numero 2 ') # pedimos el segundo número
```



```
suma = int(a) + int (b)  # calculamos la sumas
print (suma)  # imprimimos su valor
```

Ya que hemos visto que hay algo parecido a “sumar” cadenas vamos a ver que ¡también podemos multiplicarlas! ...

```
cadena = 'hola'
queSera = cadena*10
print(queSera)
```

Funciones aritméticas

Función	Definición
round	Redondeo: de un valor decimal al menor entero
abs	Valor absoluto
divmod(dividendo,diviso)	Devuelve 2 valores: cociente entero y resto
pow(base,exponente)	base elevado a exponente
cociente, resto = divmod(10,7)	

Excepciones

En los cálculos matemáticos también se pueden producir excepciones que debemos tratar de la misma forma que en las conversiones.

Por ejemplo al dividir por 0 se produce una excepción de tipo **ZeroDivisionError** que si no capturamos con un **try/except** detendrá nuestro programa

```
a = 0
b = 0
c = a / b
```

String (cadenas)

Existen multitud de operaciones que podemos hacer sobre una cadena. Veamos algunas:

len - Longitud

Podemos ver la longitud de una cadena con la función **len()**

```
palabra = input('Introduzca una palabra')
print('la palabra: ' + palabra + ' mide ')
print(len(palabra))
```

replace - Reemplazar

Sustituye todas las repeticiones de una cadena en otra dada, sin cambiar la original.

```
cadena = 'parapapa'
nueva_cadena = cadena.replace('a','e')
print(cadena + ' -> ' + nueva_cadena)
```

strip - limpiar

Elimina los caracteres "Espacio, tabuladores y " del principio y final de una cadena.

```
cadena = ' hola esto es una prueba '
nueva_cadena = cadena.strip()
print(cadena + ' -> ' + nueva_cadena)

cadena = ' hola esto es una prueba\nDe varias líneas\n'
nueva_cadena = cadena.strip('\n')
print(cadena + ' -> ' + nueva_cadena)
```

También podemos hacer que se limpie sólo la parte izquierda con **rstrip** o la derecha con **rstrip**

lower y upper- Minúsculas y mayúsculas

Las funciones **lower** y **upper** convierten a minúsculas y mayúsculas todos los caracteres.

También existen **capitalize**, **title** y **swapcase** que se deja como ejercicio averiguar su utilidad...

Formato

Hay veces que nos interesa generar una salida por print en la que se mezclan diferentes tipos de variables

A medida que ha ido evolucionando Python, han ido apareciendo diferentes formas, pero a día de hoy la más utilizada es la que se conoce como **f-string** (desde Python 3.6). Indicamos que vamos a usarla anteponiendo una **f** a la comilla inicial, y definimos el hueco en el que se pondrá la variable insertando la variable rodeada de unas llaves **{variable}**. No es necesario que hagamos conversión de la variable a cadena.

```
nombre = 'Pepe'
edad = 20

print(f'{nombre} tiene {edad} años')
```

Dentro de las f-string se pueden incluir expresiones

```
print(f'{nombre} tiene {edad} años, naciste en {2021-edad}')
```

También podemos alinear decimales y números de la siguiente manera:

- Usaremos “:” después de la variable indicando el formato
- número de cifras totales (enteras + decimales) y tras un punto “.” el número de decimales.
- Terminaremos la expresión con una “f”

```
valor1 = 1.45
valor2 = 45.45
print(f'{valor1:6.3f}')
print(f'{valor2:6.3f}')
```

```
1.450
45.450
```

Por si los encuentras en el código, otros formatos más antiguos pero que dan el mismo resultado son:

```
print("{} tiene {} años".format(nombre,edad))
print("%s tiene %d años"%(nombre,edad)) # %s indica que nombre es cadena y %d
que edad es entero
```

Unicode

Podemos acceder a cualquier carácter Unicode, si conocemos su código usando la función **chr(código)**

Por ejemplo

```
>>>chr(0x1F415)
'🐶'
>>> chr(0x1F30E)
'🌍'
>>> chr(0x1F5A5)
'💻'
>>> chr(0xFE0F)
''
>>> chr(0x1F4BD)
'💿'
```

Aprende a programar Python



Logo Python

Versión 0.99.2



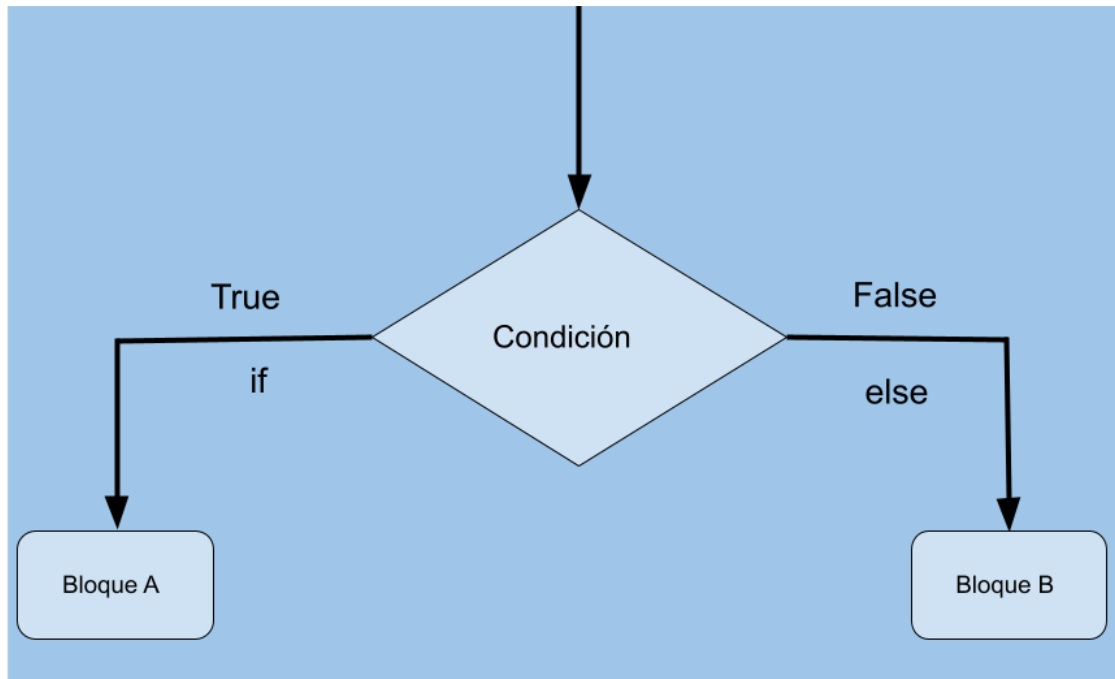
Licencia CC by SA by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Octubre 2022

Sentencias condicionales



Sentencias condicionales

En ocasiones nos interesa que determinadas sentencias de nuestro programa se ejecuten o no dependiendo del valor de alguna variable o de relación entre variables.

En estos casos usamos sentencias condicionales. La más simple es la sentencia **if** que irá seguida de una comparación que debe de dar como resultado un valor booleano y después pondremos ":". En caso de que este valor sea **True** se ejecutará el bloque de líneas que sigue a la sentencia. Si es **False** no se ejecutará.

Recordemos que para definir un bloque haremos que las líneas de código empiecen con 4 espacios (no es obligatorio que sean 4, pero es lo recomendado) más a la derecha. **Es fundamental que respetemos el formato de las líneas.**

Más adelante vamos a hablar de las posibles comparaciones, pero ahora mismo vamos a usar una comparación entre una variable y un número, y veremos cuál es mayor:

```
strEdad = input('¿Qué edad tiene? ')
edad = int(strEdad) # Convertimos a entero para comparar
if edad > 65: # ¿es el valor de edad mayor que 65?
    print('Usted puede jubilarse')
```

Llamamos **operadores relacionales** a los que usamos para comparar valores entre sí. Todos ellos dan un resultado booleano: **True** o **False**

Operador	Descripción
>	Mayor que
<	Menor que
>=	Mayor o igual
<=	Menor o igual
==	Igual
!=	Distinto

¡¡Cuidado con el operador “igual” que se representa con “dos signos igual” “==”!! No confundirlo con la asignación que usa sólo uno “=”.

Del mismo modo podemos usar comparaciones entre cadenas, comparando el valor de la variable con una cadena o dos variables entre sí.

Vamos a hacer un ejemplo sencillo de lo que sería un interface de comandos para un ordenador, donde el usuario introduce un comando y simulamos lo que el ordenador tendría que hacer con sentencias *print*:

```
# Simulación de consola de comandos v1
comando = input('Introduzca el comando: ')
if comando == 'Apagar':
    print('Apagando el equipo...')
if comando == 'Encender':
    print('Encendemos el equipo')
```

También podemos comparar para “ordenar” cadenas, puesto que existe un orden alfabético en ellas:

```
miPalabra = input('Introduzca una palabra: ')
if miPalabra > 'hola':
    print('la palabra "' + miPalabra + '" se ordena después de "hola"')
if miPalabra < 'adios':
    print('la palabra "' + miPalabra + '" se ordena antes que "adios"')
```

Me gustaría resaltar el uso de los distintos tipos de comillas para poder mostrarlas en el resultado de la sentencia *print*.

Tenemos que tener presente que si al ordenar cadenas que contienen números la ordenación no se hará como esperamos, numéricamente, puesto que se va realiza carácter a carácter:

```
if '10' < '9':
    print('esto me vuelve loco')
```

Es decir, la comparación se hace por caracteres, no como un todo: se compara el carácter '1' con el carácter '9', y por eso la cadena "10" es menor que otra.

Si convertimos las cadenas a enteros todo es como esperamos:

```
if int('10') < int('9'):
    print('esto me vuelve loco')
```

Podemos almacenar el resultado de una comparación en una variable booleana, es decir, la variable booleana guardará el resultado de la comparación que podremos usar después:

```
contento = input('¿Está contento?') == "si"
if contenido == True:
    print('Está contento')
```

La variable 'contento' tendrá un valor booleano True o False, según la respuesta.

No me canso de repetir la diferencia entre el operador *comparación de igualdad* "==" y el operador *asignación* "=".

Veamos ahora nuestro interface de comandos usando estas variables booleanas:

```
# consola de comandos v1.5
comando = input('Introduzca el comando: ')
bApagado = comando == 'Apagar'
bEncender = comando == 'Encender'

if bApagado == True:
    print('Apagando el equipo')
if bEncender == True:
    print('Encendiendo el equipo')
```

Sentencias "else" y "elif"

En muchas ocasiones tenemos un bloque de instrucciones para el caso que se cumpla y otro para cuando no se cumpla. En ese caso incluiremos la sentencia **else** : para delimitar el bloque que se ha de ejecutar en caso de que no se cumpla la condición:

```
strEdad = input('¿Qué edad tiene? ')
edad = int(strEdad)
if edad > 65: # ¿es el valor de edad mayor que 65?
    print('Usted puede jubilarse')
else: # no se cumple la condición
    print('Usted NO puede jubilarse')
```

También se da a veces que existen varios casos excluyentes entre sí y que queremos hacer comparaciones para todos ellos. En ese caso usamos **elif** que equivale a un “else if”. Veamos un caso en el ahora comparamos como el operador “menor que”:

```
strEdad = input('¿Qué edad tiene? ')
edad = int(strEdad)
if edad < 12:
    print('niño')
elif edad < 17 : # No es menor de 12 y sí menor de 17
    print('adolescente')
elif edad < 25 : # No es menor de 17 y sí menor de 25
    print('joven')
elif edad < 50 : # No es menor de 25 y sí menor de 50
    print('una buena edad')
elif edad < 65 : # No es menor de 50 y sí menor de 65
    print('casi abuelete')
else: # es mayor o igual de 65
    print('Usted puede jubilarse')
```

Conviene fijarse en este ejemplo que no funcionaría igual si pusiéramos sólo sentencias if puesto que cuando un valor es menor que 25 también lo es que 50 y los sucesivos...

Ahora podemos hacer más eficiente nuestro interface de comandos, puesto que sólo hacemos las comparación hasta que se encuentra el comando solicitado y ya se dejan de hacer más comparaciones:

```
# Simulación de consola de comandos v2
comando = input('Introduzca el comando: ')
if comando == 'Apagar':
    print('Apagando el equipo...')
elif comando == 'Encender':
    print('Encendemos el equipo')
else :
    print('Comando ' + comando + ' no reconocido')
```

Operadores lógicos

A veces tenemos que comprobar varias expresiones combinando los resultados de todas ellas en un único resultado.

Para ello usaremos los operadores lógicos

Operador	Resultado
and	todos los valores han de ser <i>True</i>
or	al menos uno de los valores ha de ser <i>True</i>

not

se invierte el resultado

Podemos usar los operadores lógicos para encadenar varias comparaciones como vemos en este ejemplo que determina *si un año es bisiesto* para lo que hay que ver si divisible por 4, pero no múltiplo de 100 salvo que lo sea de 400 (sí, es un poco lioso, pero así es como se hace...)

Código

```
# Programa que determina si un año es o no bisiesto
# Divisible por 4
# No divisible por 100 salvo que
# Sea divisible por 400
year = int(input('Introduzca el año: '))

if (year%4)==0 and ( (year%400)==0 or not ((year%100)== 0)):
    print('Es bisiesto!!')
else:
    print ('No es bisiesto!!')
```

Otra forma de hacer varias comparaciones es usando notación matemática, por ejemplo, para comprobar si una variable está dentro de un rango.

Normalmente haríamos:

```
if minimo < variable and variable < maximo :
    ...
```

Usando notación matemática queda más claro y compacto:

```
if minimo < variable < maximo :
    ...
```

Sentencias anidadas

A veces la decisión es compleja y necesitamos poner unas condiciones dentro de otras. Decimos entonces que las condiciones están anidadas. En este caso tenemos que extremar el cuidado con la indentación de las líneas.

Vamos a hacer un ejemplo que nos da los días que tiene un mes dado. Convertimos directamente el valor que nos da el usuario a entero, para poder hacer las comparaciones entre números.

En el if/else más externo comprobamos si el valor del mes es correcto, en los interiores según los valores de los meses:

Código

```
# Nos da los días que tiene el mes seleccionado
mes = int(input('Introduce el mes:'))
year = int(input('Introduce el año:'))
# Comprobamos si está entre 1 y 12
if 1 <= mes <= 12:
    if mes == 2:
        if (year%4)==0 and ( (year%400)==0 or not ((year%100)== 0)):
            dias = 29
        else:
            dias = 28
    elif (mes == 4) or (mes == 6) or (mes == 9) or (mes == 11):
        dias = 30
    else:
        dias = 31
    print(f'El mes {mes} del año {year} tiene {dias} días')
else:
    print('El mes debe ser entre 1 y 12')
```

Vamos a aprovechar lo visto para dar un nivel de seguridad a nuestro interface de comandos. Para ello preguntaremos el nombre al usuario y sólo si el usuario es adecuado daremos acceso a los comandos. Además si quiere apagar el ordenador le pediremos una confirmación:

Código

```
# Simulación de consola de comandos v3
usuario = input('Introduzca el nombre de usuario: ')
if usuario == 'pepe' or usuario == 'admin':
    print('Bienvenido ' + usuario)
    comando = input('Introduzca el comando: ')
    if comando == 'Apagar':
        confirmacion = input('Confirme que desea apagar respondiendo "Si": ')
    )
    if confirmacion == 'Si':
        print('Apagando el equipo...')
    else:
        print('No se ha confirmado el apagado')
    elif comando == 'Encender':
        print('Encendemos el equipo')
    else :
        print('Comando ' + comando + ' no reconocido')
else:
    print('Acceso denegado')
```

Vamos a fijarnos en los diferentes niveles de anidamiento de las sentencias y cómo hemos utilizado el operador lógico “or” para comprobar entre los nombres de usuario.

Aprende a programar Python



Logo Python

Versión 0.99.2



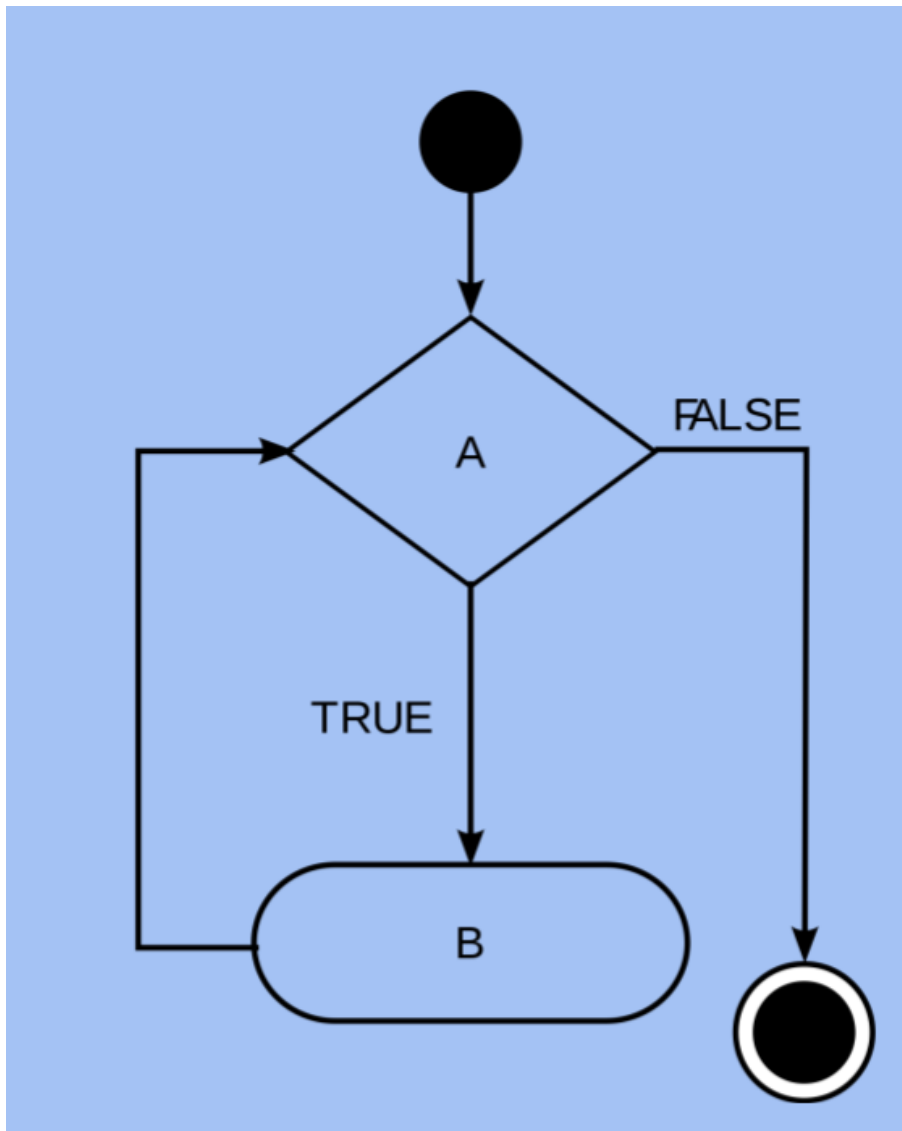
Licencia CC by SA by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Octubre 2022

Bucles



Bucle

En todos los programas necesitamos que partes de nuestro código se repitan. De hecho todos los programas/aplicaciones tienen una parte que llamamos bucle principal (main loop) que es donde el programa se pasa casi todo el tiempo.

Un bucle siempre está formado por un bloque de código que se repite y por una condición que decide si ha de repetirse una vez más o se termina.

Tenemos 2 formas de hacer un bucle en Python:

- Bucle **while** cuando tenemos claramente definida la condición que se debe cumplir para que se repita

- Bucle **for** para cuando tenemos claramente definidas las repeticiones que vamos a hacer o sabemos los elementos a los que vamos a aplicar nuestro código: Por ejemplo vamos a revisar todas las palabras de un texto, o todas la letras de una palabra.

En cualquier caso, en muchas ocasiones se pueden usar los dos con pequeñas diferencias.

Podemos detener un bucle usando en cualquier momento **Ctrl + C**

Bucle while

Hemos dicho que está indicado cuando una condición nos dice que repetir algo. Pondremos después de la palabra reservada **while** la condición de repetición, seguida de ":" que marcarán el comienzo del bloque que se va a repetir, con la indentación adecuada.

En este primer ejemplo vamos a contar hasta 100, imprimiendo los números. Usaremos una variable que hará como contador, que imprimiremos e incrementaremos a cada paso. La condición será que ese contador sea menor o igual a 100

```
# imprimimos los 100 números
contador = 1
while contador <= 100:
    print(contador)
    contador += 1
```

Vamos a modificar ahora el ejemplo anterior para que muestre sólo los números que son múltiplos de 7 hasta el número 100, para lo que añadimos una sentencia condicional que sólo mostrará el valor de contador si es múltiplo de 7.

Vemos cómo hemos creado distintos bloques de código con la correspondiente indentación.

```
# imprimimos los múltiplos de 7 hasta 100
contador = 1
while contador <= 100:
    if contador % 7 == 0:
        print(contador)
    contador += 1
```

Salir del bucle con break

A veces necesitamos detener la repetición de un bucle aunque la condición de repetición se siga cumpliendo. Para ello usaremos la palabra reservada (keyword) **break** que saldrá del bucle inmediatamente

Vamos a hacer ahora un programa que se detenga cuando encuentre el primer múltiplo de 77. Por tanto añadimos dentro del bloque la condición if de la sentencia **break**, que detendrá la ejecución.

Al tener dos maneras de terminar el bucle (porque ya no se cumpla la condición del while y porque llamamos a break), necesitamos una manera de distinguirlas. Para ello podemos añadir un bloque **else** que sólo se ejecutará si hemos salido porque ya no se cumplía la condición del while.

```
# imprimos el primer múltiplo de 77
contador = 1
while contador <= 78:
    if contador % 77 == 0:
        print(contador)
        break
    contador += 1
else:
    print('No hay multiplo')
print('Hemos terminado')
```

Podemos ver los diferentes comportamientos cambiando el valor del número hasta el que llegamos buscando el múltiplo de 77: llegando hasta 75 se mostrará la sentencia del else, mientras que si llegamos más allá del 77 se mostrará la línea anterior al break.

Bucles infinitos

En algunas ocasiones no sabemos realmente la condición para detener un bucle, en ese caso diremos que hacemos un bucle infinito y saldremos del mismo por medio de una sentencia **break**

Vamos a hacer un programa que busque el múltiplo de 77 más cercano a 1000 y que sea menor. Para ello empezaremos en 1000 e iremos bajando hasta encontrarlo.

Como no sabemos a priori dónde para, pondremos una condición en while que se cumpla siempre: **while True** y pararemos con el break

```
# buscar múltiplo de 77 más cercano y menor que 1000
```

```
contador = 1000
```

```
while True: # bucle infinito
    if contador % 77 == 0:
        print(contador)
        break
    contador -= 1
```

Otra manera de hacerlo es utilizando una variable booleana en la condición, que valdrá True hasta que encontremos el número. De esta manera no tendremos que usar **break**

```
# buscar múltiplo de 77 más cercano y menor que 1000
```

```
contador = 1000
```

```
repetir = True
```

```
while repetir:
    if contador % 77 == 0:
        print(contador)
        repetir = False
    contador -= 1
```

Realmente, en este se podría hacer el bucle de una manera diferente para que la condición incluya la sentencia que va a desencadenar el break, pero en caso más complejos no va ser posible.

```
# buscar multiplo de 77 mas cercano y menor que 1000
```

```
contador = 1000
```

```
while contador % 77 != 0:
    contador -= 1
else:
    print(contador)
```

Tenemos que tener cuidado con las condiciones para salir de los programas porque pudiera darse el caso en que éstas no se dieran nunca y el bucle no parara nunca, como es el caso de este programa

```
# contador de 2 en 2 hasta 100
```

```
contador = 1
```

```
while True: # bucle infinito
```

```
if contador == 100: # Paramos al llegar a 100
    break
print(contador)
contador += 2
```

Para evitarlo sólo tenemos que modificar la condición de parada por una que nos lo asegure como sería **contador >= 100**

Bucle for

Hemos dicho que el bucle **for** está pensado para recorrer conjuntos de cosas, como pueden ser palabras en una frase o las letras de una palabra.

Por ello vamos a empezar haciendo un programa que nos deletrea una palabra

```
# deletrear
```

```
cadena = input ()
for letra in cadena:
    print(letra)
```

Dentro del bucle **for** también podemos usar **break** igual que en **while**

Vamos a hacer ahora un programa que nos muestra todos los caracteres menos uno dado, la 'l' en estaco

```
# deletrea palabras
cadena = input('Introduzca una palabra ')

for caracter in cadena:
    if caracter != 'l':
        print(caracter)
```

Tanto en el bucle **for** como el **while** tenemos un recurso para hacer que dentro de una iteración no se ejecuten algunas de las sentencias del bloque. Es la sentencia **continue** que pasará a la siguiente iteración saltándose las líneas del bloque que queden de esa iteración.

Usándola el programa anterior quedaría:

```
# deletrea palabras
cadena = input('Introduzca una palabra ')

for caracter in cadena:
    if caracter == 'l':
        continue # repetimos desde aquí
    print(caracter)
```


Range

Podemos usar también el bucle **for** cuando queremos iterar sobre números usando la función **range()** (rango). Necesitaremos una variable que actúe de contador que será la que vaya recorriendo el rango.

A la función **range** le tenemos que indicar al menos 2 números: el inicio y el final, teniendo en cuenta que el final no se incluirá. Es decir, `range(1,100)` terminará en 99.

El programa para mostrar los números hasta 100 sería

```
# contador hasta 100
```

```
for contador in range(1,101):  
    print(contador)
```

También podemos darle un tercer argumento que será el incremento entre pasos, que por defecto es uno. Por ejemplo `range(0,100,2)` recorrerá los números pares desde el 0 hasta el 98

También podemos usar incrementos negativos, con lo que el rango se recorrerá en sentido descendente `range(100,1,-1)`

Y si queremos mostrar desde el 100 hasta el 1

```
# contador hasta 100
```

```
for contador in range(100,0,-1):  
    print(contador)
```

Podemos omitir el inicio de `range` si éste es 0, lo que nos permite simplificar. Por ejemplo si queremos repetir algo N veces, podemos hacer `for i in range(N)`:

Bucles anidados

En algunas ocasiones necesitamos usar 2 bucles, uno dentro de otro para recorrer todas las posibilidades, por ejemplo si quisiéramos escribir las tablas de multiplicar enteras, necesitaríamos un bucle que cambia con cada tabla (la 1, la del 2, ...) y otro que cambia con el número dentro de cada tabla

Decimos que los bucles compuestos de esta manera están **anidados**.

Vamos a hacer el programa que nos muestra la tabla de multiplicar. Para recorrer la tabla usaremos la variable **i** (que vemos que está en el bucle exterior) y la **j** será la que recorra dentro de cada tabla.

Añadiremos una indicación de la tabla que se está mostrando en cada momento.

tabla de multiplicar

```
for i in range(1,11):
    print(f'\nTabla del {i}\n-----')
    for j in range(1,11):
        print(f'{i} x {j} = {i*j}')
```

Ejemplos

Bucle while

Vemos en este ejemplo otra manera de calcular múltiplos

cálculo de múltiplos de 5 entre 1 y 1000

```
numero = 5
while numero<1000:
    numero = numero + 5
    print(numero)
```

Ejemplo while: 5.0 Múltiplos

Ejemplo bucle for

Vamos a ver en este ejemplo cómo podemos calcular a la vez si son múltiplos de 5 y de 7 usando un bucle **for**

cálculo de múltiplos de 5 y/o 7 entre 1 y 1000 usando bucle for

```
for numero in range(1, 1001):
    bMultiplo5 = numero % 5 == 0 # comprobamos si es múltiplo de 5
    bMultiplo7 = numero % 7 == 0 # comprobamos si es múltiplo de 7
    if bMultiplo5 and bMultiplo7 :
        print(f' {numero} es múltiplo de 5 y de 7')
    elif bMultiplo5 :
        print(f' {numero} es múltiplo de 5')
    elif bMultiplo7 :
        print(f' {numero} es múltiplo de 5 y de 7')
```

Ejemplo bucle for v1

Veamos ahora otra forma de hacerlo

cálculo de múltiplos de 5 y/o 7 entre 1 y 1000 usando bucle for

```
for numero in range(1, 1001):
```

```

bMultiplo5 = numero % 5 == 0 # comprobamos si es múltiplo de 5
bMultiplo7 = numero % 7 == 0 # comprobamos si es múltiplo de 7
strMensaje = ''
if bMultiplo5 or bMultiplo7 :
    strMensaje = f' {numero} es múltiplo'
    if bMultiplo5 :
        strMensaje += ' de 5'
    if bMultiplo7 :
        strMensaje += ' de 7'
    print(strMensaje)

```

Ejemplo bucle for v2

Suma de números hasta N

Vamos a hacer ahora un mismo ejemplo usando los dos tipos de bucles. Calcularemos cuando vale la suma de todos los enteros hasta uno dado, N.

Para hacer el cálculo usaremos una variable que recorra el rango 1 hasta N, que generaremos con `range(1,N+1)` y otra que vaya acumulando la suma de todos los anteriores.

```

# Sumas de enteros has N
N = int(input('¿Numero? '))
suma = 0
for sumando in range(1,N+1):
    suma += sumando

print(f'La suma hasta {N} es: {suma}')
print(f'Algebraicamente es {N*(N+1)//2}')

```

codigo

Vamos a comprobar el resultado con el cálculo algebraico que sabemos que es $N*(N+1)/2$

Si ahora hacemos el programa con while, nos va a quedar muy parecido.

```

# Sumas de enteros has N
N = int(input('¿Numero? '))
suma = 0
sumando = 1
while sumando <= N:
    suma += sumando
    sumando += 1

print(f'La suma hasta {N} es: {suma}')
print(f'Algebraicamente es {N*(N+1)//2}')

```

José Antonio Vacas Martínez

Aprende a programar Python



Logo Python

Versión 0.99.2



Licencia CC by SA

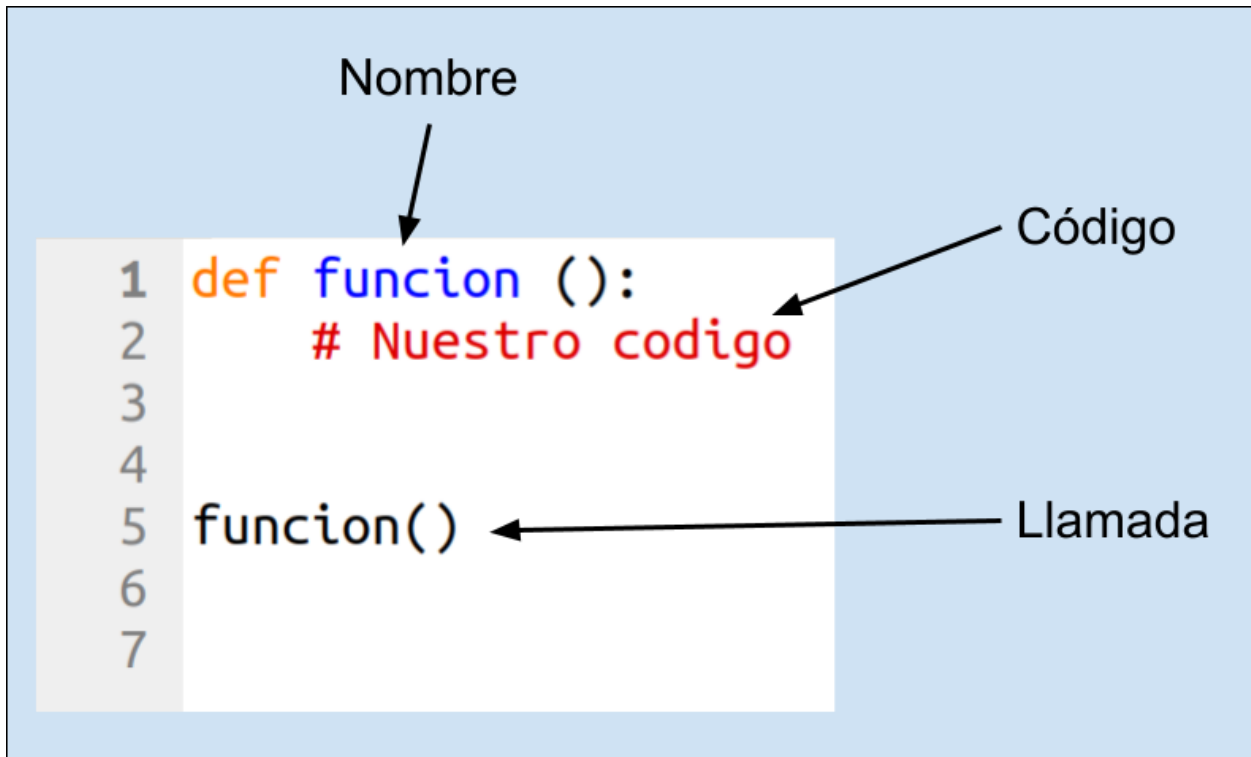
by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Septiembre 2022

Funciones



Una función es una estructura que nos va a permitir usar varias un bloque de código sin tener que repetirlo.

La función tendrá un nombre y antes de usarla tenemos que definirla, lo que se hace precediendo el nombre de la función de la palabra reservada **def** y tras unos paréntesis pondremos ":" que nos indican que a continuación sigue el bloque de código de la función. Pondremos el bloque de código indentado para dejar claro las líneas que lo forman.

Para usar la función, sólo tenemos que llamarla, lo que se hace simplemente poniendo su nombre y los paréntesis.

Veamos un ejemplos sencillo de una función llamada "saludo" y que nos mostrará nuestro famoso "Hola Python":

```
def saludo():
    """
    Muestra un saludo en pantalla
    """
    print('Hola Python')
```

La ejecutaremos con:

```
>>> saludo()  
>>> Hola Python
```

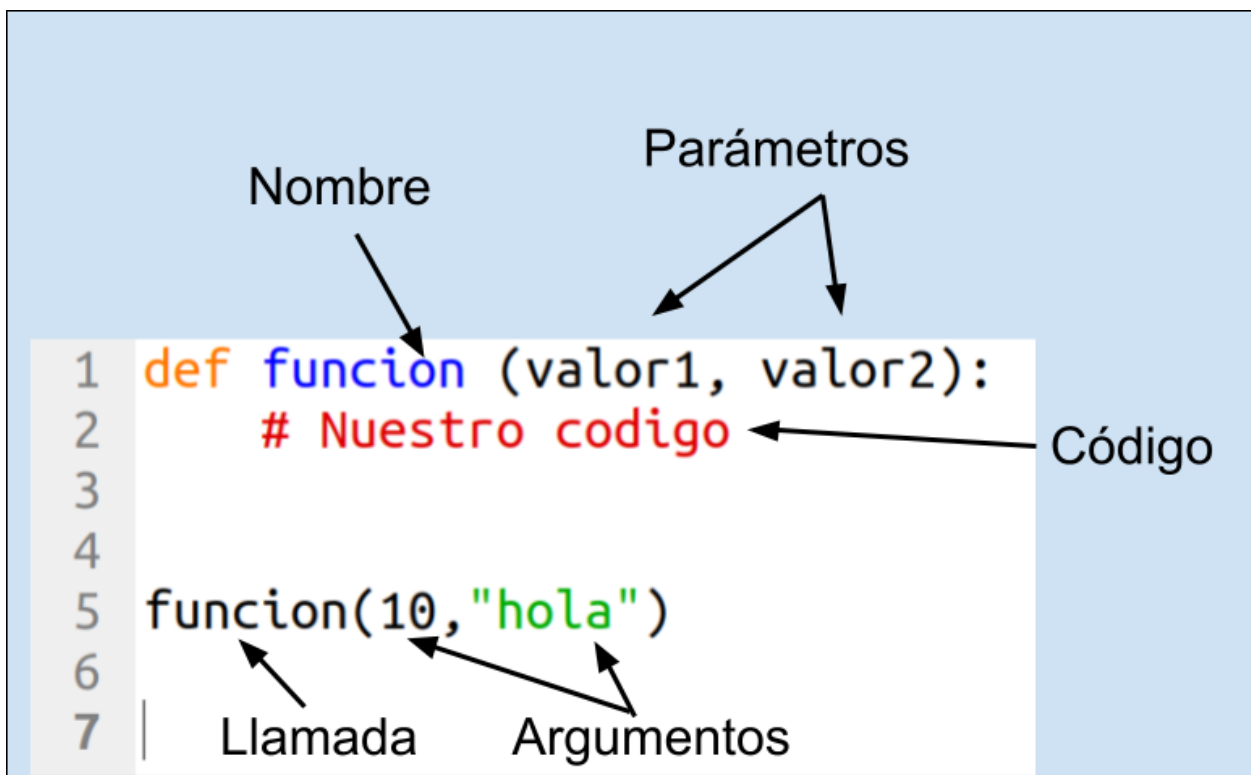
Si documentamos la función incluyendo un comentario extenso al principio, además de incluir información de su uso, permitimos acceder a esa información desde el comando **help**. Ahora si usamos **help(saludo)**, no dará la información que hemos incluido en el comentario.

Parámetros y argumentos

Ya que tenemos una función que saluda, sería una buena idea que nos permitiera un poco más de flexibilidad al saludar, permitiéndonos hacer un saludo a cualquier persona.

Para ello vamos a añadir lo que se conoce como un **parámetro**, que es una variable que incluimos entre los paréntesis de la definición y que podremos usar dentro de nuestro código.

Llamamos parámetro a las variables que usamos en la función y argumentos a los valores que damos a esas variables cuando llamamos a la función para que esta se ejecute.



Veamos un ejemplo de una función con un parámetro y como para llamarla tenemos que pasarle un argumento:

```
def saludo(nombre):  
    '''  
    Muestra un saludo en pantalla  
  
    param nombre: a quien saludamos  
    '''  
    print(f'Hola {nombre}')
```

```
>>> saludo('Juan')
```

Argumentos no obligatorios

A veces queremos evitar tener que dar valor a un parámetro necesariamente, pero sí que queremos tener la flexibilidad de poder usar distintos argumentos.

En este caso podemos hacer que los parámetros tengan un valor por defecto que se usará si no damos ningún argumento. Lo indicaremos dando un valor al parámetro en la definición.

Nuestra función saludo quedaría así si le ponemos al parámetro nombre el valor "Python" por defecto:

```
def saludo(nombre = 'Python'):  
    '''  
    Muestra un saludo en pantalla  
  
    param nombre: a quien saludamos  
    '''  
    print(f'Hola {nombre}')
```

```
>>> saludo('Juan')  
>>> Hola Juan  
>>> saludo()  
>>> Hola Python
```

Funciones en Python

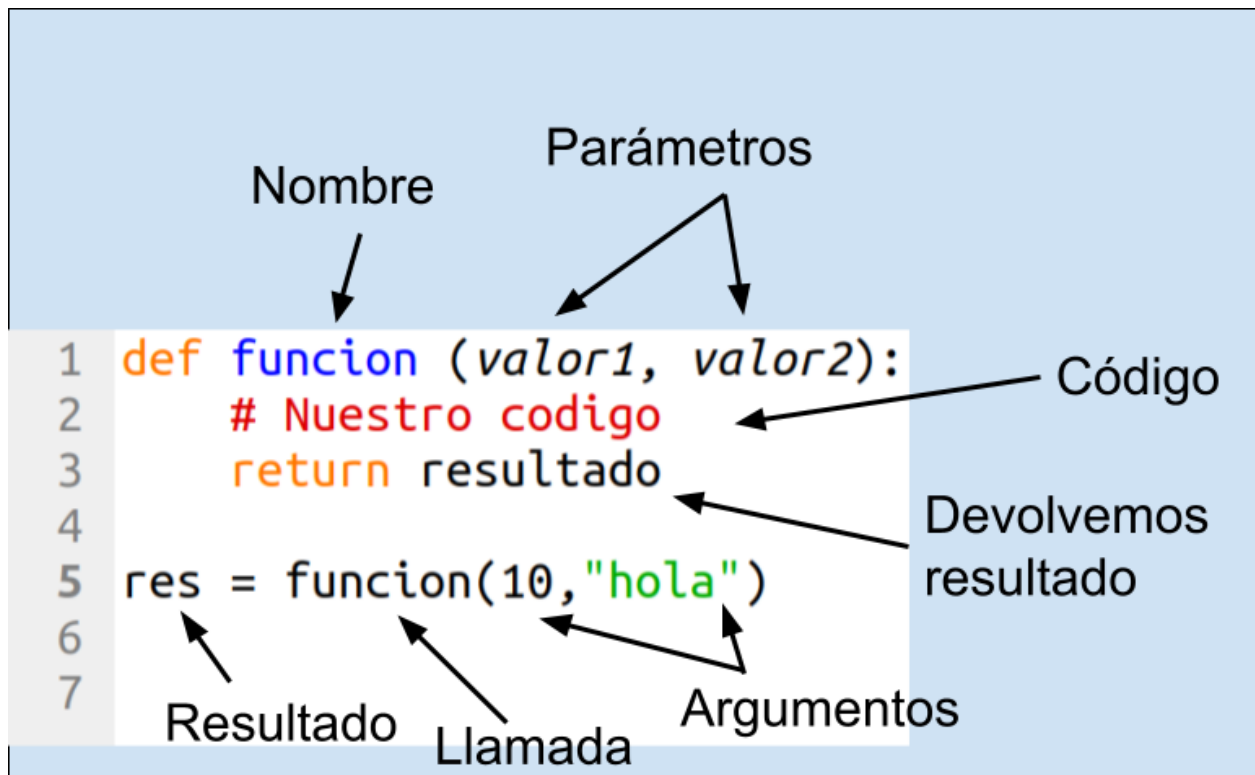
Ahora que sabemos lo que son, nos damos cuenta de que llevamos tiempo usando funciones en Python, como son "print", "input", "len", help,

De hecho a lo largo del curso vamos a seguir viendo multitud de funciones y muchas más que crearemos nosotros.

Funciones que devuelven valores

Algo muy frecuente es que una función devuelva un valor, un ejemplo puede ser la función **input()** o la función **len()** que ya hemos utilizado.

Para devolver un valor sólo tenemos que añadir como última sentencia de la función la palabra reservada **return** seguida del valor que queremos devolver.



Vamos a hacer como ejemplo una función que devuelve la suma de 2 valores:

```
def suma(sumando1, sumando2):  
    '''  
    suma: devuelve la suma de 2 valores  
  
    param sumando1: primer sumando  
    param sumando2: segundo sumando  
  
    devuelve el resultado de sumar los dos valores  
    '''  
    resultado = sumando1 + sumando2  
    return resultado
```


Indicación de Tipos en argumentos

Como ya vimos, a partir de la versión 3 de Python, podemos indicar el tipo que va a tener una variable. También podemos indicar el tipo que tendrá un argumento y el tipo de variable que devolverá la función.

```
>>> def saludo(nombre: str) -> str:
>>> # Esperamos un argumento de tipo str y devolvemos un str
>>>     return 'Hola ' + nombre
>>> saludo('pepe')
'Hola pepe'
>>> print(saludo('pepe'))
Hola pepe
>>> print(saludo(7))
Traceback (most recent call last):
  File "<pyshell>", line 1, in <module>
    File "<pyshell>", line 2, in saludo
TypeError: must be str, not int
```

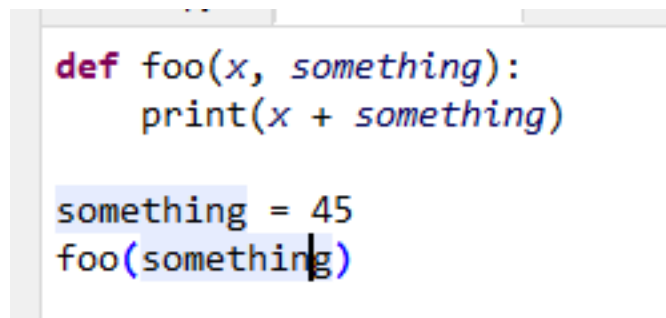
Como vemos en el ejemplo en el caso de que no sea válido el tipo se produce una Exception de tipo `TypeError`.

Variables locales y variables globales

Las variables que llamamos **globales** son aquellas que están definidas en todo el ámbito del programa y que nos van a permitir utilizarla en cualquiera de las funciones

Por defecto todas las variables que declaramos en funciones son **locales**, es decir, solamente están definidas dentro de la función en la que estamos trabajando. Podemos utilizar una de estas variables como global sin más que anteponer la palabra **global** al nombre de esta variable.

En esta imagen vemos que el editor nos muestra, resaltando, lo que entiende que son una misma variable:



```
def foo(x, something):
    print(x + something)

something = 45
foo(something)
```

La zona donde podemos usar una variable se llama **ámbito** (scope en inglés) y para evitar errores es muy importante tener claro cuál es en cada caso.

Vamos a ver todo esto con un ejemplo donde vemos que aunque tenemos 2 variables llamadas igual, no son la misma, si no que la variable que está en la función actúa de modo local, diremos que es una **variable local** y su ámbito será la función.

La variable que está definida fuera de la función diremos que es una **variable global** y que se puede usar en cualquier parte del programa.

```
def saludo(nombre = 'Python'):
    """
    Muestra un saludo en pantalla

    param nombre: a quien saludamos
    """
    print(f'Hola {nombre}')

nombre = 'Pepe'
saludo('Juan')
print(nombre)
```

Si abrimos la pestaña **Variables**, veremos que Thonny identifica a una de ellas como local y a otra como global.

Para que Python interprete las dos como una misma variable debemos indicarlo incluyendo dentro de la función la referencia **global** delante de la variable idioma:

```
def saludo(nombre = 'Python'):
    """
    Muestra un saludo en pantalla

    param nombre: a quien saludamos
    """
    global idioma
    idioma = 'en'
    if idioma == 'es':
        print(f'Hola {nombre}')
    else:
        print(f'Hello {nombre}')

idioma = 'es'
saludo('Juan')
```

En este caso también tenemos la alternativa de hacer que la función **saludo** tenga un parámetro "idioma" que tiene un **valor por defecto**:

```
def saludo(nombre = 'Python', idioma = 'en'):
```

Muestra un saludo en pantalla

param nombre: a quien saludamos
'''

```
if idioma == 'es':
    print(f'Hola {nombre}')
else:
    print(f'Hello {nombre}')
```

```
idioma = 'es'
saludo('Juan',idioma)
saludo(nombre='Manolo', idioma='en')
saludo(idioma='es', nombre='Felipe')
```

Como ya hemos visto, dado que los parámetros tienen nombre, podemos usarlos en el orden que queramos, siempre que indiquemos el parámetro al que queremos dar cada argumento.

Funciones recursivas

Vamos a hacer ahora un programa para calcular un concepto matemático que se llama **factorial** y que se usa mucho en cálculos de Combinatoria.

Para calcular $N!$, el factorial de N , tenemos que calcular el producto de todos los números enteros desde 1 hasta N .

Por ejemplo: el factorial de 5, que se representa con $5!$ será $5! = 5 * 4 * 3 * 2 * 1$ que valdrá 120.

Vamos a hacer una función que lo calcule usando un variable *resultado* para ir calculando el producto y un bucle *for* que iteraremos entre 1 y N

```
def factorial(N):
    '''
    calcula factorial: productos de los enteros hasta el 1
    5! = 5*4*3*2*1

    param N: número entero
    '''
    resultado = 1 # iremos guardando el resultado parcial
    for numero in range(N,1,-1): # vamos desde N hasta 1
        resultado *= numero
    return resultado
```

Si lo probamos

```
>>> factorial(5)
>>> 120
```

Si le damos un poco de vueltas al concepto, veremos que $5! = 5 * 4!$, es decir, podemos calcular el factorial de N en función del factorial de $N-1$, y este a su vez en función del factorial de $N-2$, y así sucesivamente hasta $1!$ que sabemos que es 1 .

Sería como desde dentro de la función factorial, volver a llamar a la función factorial pero con el número anterior como argumento y así hasta que al llegar al llamarlo con el argumento 1 devolveremos el valor 1 .

Una implementación sería como la siguiente:

```
def factorial_recursivo(N):
    """
    calcula factorial: productos de los enteros hasta el 1
    5! = 5*4*3*2*1 = 5 * 4!

    param N: numero entero
    """
    if N == 1:
        resultado = 1
    else:
        resultado = N * factorial_recursivo(N-1)
    return resultado
```

Si comprobamos el valor, vemos que es correcto.

```
>>> factorial_recursivo(5)
>>> 120
```

A las funciones que se llaman a sí mismas de esta manera las denominamos **funciones recursivas** y es algo que se usa bastante.

José Antonio Vacas Martínez

Aprende a programar Python



Logo Python

Versión 0.99.2



Licencia CC by SA

by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Octubre 2022

Colecciones

Una colección es una estructura que como su nombre indica nos permite almacenar varios elementos. Existen diferentes tipos de colecciones, cada una con diferentes características lo que lo hace más indicado para según qué tipo de almacenamiento.

El tipo más sencillo son las **Tuplas** (tuple), un conjunto invariable de elementos, no tienen que ser del mismo tipo.

Veremos también las **Listas** (list) nos permiten guardar también múltiples objetos y de cualquier tipo, con la peculiaridad de que están ordenados. Disponen de todos los mecanismos para trabajar dinámicamente con ellas, añadiendo o quitando elementos, reordenando.

Otro tipo muy importante son los **Diccionarios** (dict), donde tenemos un conjunto de pares que no están ordenados, formados por una clave (key) y un valor (value). La clave tiene que ser única.

También existen los **Conjuntos** (sets), que almacenan valores sin ordenar y que no pueden estar repetidos. Sólo se pueden guardar elementos inmutables (que no cambian) y que de los que se puede obtener un valor **hash** (algo similar a un id único)

Para cada caso escogeremos la estructura que nos facilite más la programación, donde la propia naturaleza de la estructuras nos será muy útil.

Por ejemplo para guardar una lista de propiedades, parece más interesante utilizar un diccionario, donde guardaremos de forma natural el nombre de la propiedad y su valor.

Si necesitamos mantener varios elementos sin que se produzca repetición, parece que un conjunto sería lo más indicado.

Tuplas

El tipo más sencillo de colección son las **Tuplas**, que son un conjunto invariable de elementos, es decir, una vez creadas no podemos modificarlas, ni añadir ni quitar elementos. Los elementos no tienen que ser del mismo tipo, ni tampoco podemos cambiarlos de valor, es decir sus elementos también tienen que ser invariantes, por lo que no pueden ser variables.

¿Qué datos podemos usar que no necesiten cambio? Por ejemplo los días de la semana:

```
>>> dias = ('L', 'M', 'X', 'J', 'V', 'S', 'D')
```

No hay ningún problema en que se repitan elementos dentro de la tupla.

Las definimos poniendo los elementos que la forman entre paréntesis:

```
>>> coordenadas = (1,3,4)
>>> elementos = ('a',7,'hola qué tal')
```

Podemos acceder a un elemento de la tupla con su índice que irá entre 0 y N-1 siendo N el número de elementos de la tupla, indicándolo entre llaves:

```
>>> elementos[0]
'a'
```

Si intentamos acceder a un elemento más allá del último obtendremos una excepción del tipo *IndexError*.

También podemos acceder a los elementos con índices negativos, en cuyo caso se entiende que índice -1 corresponde al último y accederemos hacia atrás:

```
>>> elementos[-1]
'hola qué tal'
>>> elementos[-2]
7
```

Podemos *sumar* tuplas usando el operador "+" y obtendremos una nueva con los elementos de las 2 sumadas.

Si usamos el operador multiplicación "*" lo que haremos será repetirla N veces.

También podemos acceder a una parte de la tupla usando el índice de elemento inicial y del final separados por ":", obtendremos una lista con los elementos seleccionados:

```
>>> elementos[1:2]
(7,'hola qué tal')
```

También podemos omitir el número inicial o el final si queremos que empiece en el principio o acabe al final de la tupla:

```
>>> festivos = dias[5:]
>>> laborables = dias[:5]
```

Se suele llamar rodaja (slice) a esta parte que tomamos de una colección mayor.

También podemos usar un tercer entero para indicar cada cuantos elementos queremos iterar.

¿Te recuerda a algo? Sí: la función **range** es un generador de tuplas de números.

Iterando

Podemos recorrer los elementos de una tupla usando el bucle for:

```
valores = (1,5,2,7)
for valor in valores:
    print(valor)
```

Podemos conocer el tamaño de una tupla usando el método *len*, lo que nos permite recorrer la tupla usando un bucle *while*

```
valores = (1,5,2,7)
contador = 0
while contador < len(valores):
    print(valores[contador])
    contador += 1
```

Podemos saber si un elemento está en una tupla usando la palabra reservada *in*:

```
>>> 4 in valores
False
```

Que nos dará True o False según si el elemento está o no en la tupla.

```
>>> 'X' in festivos
False
```

También podemos saber cuántas veces aparece un elemento con el método *count()*:

```
>>> valores.count(5)
1
```

Listas

Una lista es una colección en la que los elementos que pueden ser de distintos tipos, están ordenados.

Características de las listas:

- No todos los elementos tienen que ser del mismo tipo.
- La lista es dinámica y puede cambiar de tamaño, es decir, podemos añadir y quitar elementos.
- Los elementos están ordenados, en cuanto a su posición en la lista. No es que por defecto se les aplique un orden al insertarlos. Además dado que pueden ser de distintos tipos, no siempre se puede establecer un orden según su valor.

Si has programado en otro lenguaje, las listas te pueden recordar a los arrays, pero en Python son mucho más dinámicas y versátiles.

Para declararla lo haremos de la siguiente forma:

```
mi_lista = [ 10, "hola", "Pepe",25]
```


Accederemos a cada elemento usando su posición, empezando en 0 y terminando en el elemento N-1, siendo N el número de elementos de la lista.

```
print(mi_lista[2])
```

Podemos modificar el elemento que está en una posición dada:

```
mi_lista = [ 10, "hola", "Pepe",25]
```

```
print("inicialmente",mi_lista[2])
```

```
mi_lista[2] = "Juan"
```

```
print("ahora es",mi_lista[2])
```

También podemos usar un índice negativo siendo **-1** el elemento de la última posición y así hasta llegar al primero.

Si intentamos acceder a un elemento más allá de la última o de la primera obtendremos una excepción **IndexError**

Podemos crear una lista vacía con `lista = []`

SubListas

También podemos recuperar varios elementos de una lista, es lo que se llama una **sublista**. Para ello indicaremos un rango de valores, que son 2 valores numéricos separados por ":", de la siguiente forma **principio:final**, siendo **principio** el índice del primer elemento de la sublista y **final** el índice posterior al último que vamos a recuperar.

```
>>> print(mi_lista[0:2])  
[10, 'hola']
```

Si omitimos el primer valor se entenderá que nos referimos al primer valor de la lista

```
>>> print(mi_lista[:2])  
[10, 'hola']
```

y si omitimos el segundo que queremos llegar hasta el elemento final

```
>>> print(mi_lista[2:])  
['Juan', 25]
```

Si omitimos los dos tendremos la lista entera

```
>>> print(mi_lista[:])  
[10, 'hola', 'Juan', 25]
```

También podemos incluir un tercer número en la sublista (separado por otro :) que indicará cada cuantos elementos contamos al elegirlos (lo que sería el "paso"). Por defecto este número es 1, pero podría ser otro.

```
>>> print(mi_lista[::2])  
[10, 'Juan']
```

Una utilidad de este tercer parámetro es invertir el orden de nuestra lista usando **-1**. Por ejemplo:

```
>>> print(mi_lista[::-1])  
[25, 'Juan', 'hola', 10]
```

Operaciones sobre listas

Si nuestra lista está formada por elementos homogéneos (todo números o todo texto) podremos realizar una serie de operaciones sobre ellas:

- **Ordenación:** `mi_lista.sort()` o `mi_lista.sort(reverse=True)` para invertirla en orden inverso. Una vez ordenada sabremos que el primero o el último serán el mínimo o el máximo valor. Conviene recalcar que la operación se realiza sobre la propia lista, no es que la llamada nos devuelve una lista ordenada.
- Obtener el máximo aplicando la función **max** `max(mi_lista)`.
- Obtener el mínimo aplicando la función **min** `min(mi_lista)`.
- Calcular su longitud con **len** `len(mi_lista)`.

Búsquedas

Una funcionalidad que usaremos muchos son las búsquedas, que podremos hacer con la palabra reservada **in**. Obtendremos como resultado un booleano, siendo *True* que se ha encontrado y *False* en caso contrario.

Conviene destacar que se distingue entre mayúsculas y minúsculas.

```
>>> mi_lista = ['Juan', 'Dionisio', 'Pepe', 'Manolo']  
>>> 'Juan' in mi_lista  
True
```

Una vez que sabemos que el elemento está en la lista podemos preguntar en qué posición con **index**:

```
>>> posicion = mi_lista.index("Juan")  
0
```

Si intentamos buscar un elemento que no está se produce una excepción de tipo **ValueError**.

También podemos contar el número de veces que se repite un valor con **count** :

```
>>> mi_lista.count("Juan")
1
>>> mi_lista.count("juan")
0
```

Como vemos podemos usar la función *count* para ver si un elemento está o no en una lista.

Edición en listas

Hemos visto cómo cambiar, buscar, contar o acceder a elementos de una lista. Pero se pueden hacer más operaciones como las siguientes, a las que llamaremos a partir de nuestra lista original

- **append(elemento)**: Añade un elemento al final de la lista.
- **clear()**: borra todos los elementos de una lista.
- **insert(posicion, valor)**: inserta en *posicion* el valor indicado, desplazando los siguientes.
- **remove(elemento)**: elimina **elemento** de la lista. Para esto mismo podemos usar el "arcaísmo" **del**:

```
del lista[4]
```

- **copy()**: devuelve una copia de la lista. Podemos usarlo para crear una nueva si no queremos alterar la original.
- **reverse(lista)**: invierte el orden de la lista.
- **pop()**: Devuelve el último elemento de la lista y lo retira de la lista. También podemos usarlo para eliminar un elemento indicando el índice a sacar de la lista.

```
lista.pop(2)
```

Esta función nos muestra que podemos usar una lista como soporte para un funcionamiento más complejo como sería una cola o pila, donde vamos eliminando el último elemento.

- **extend(lista2)**: añade a la lista actual el contenido de lista2. Es equivalente a hacer una suma de las dos listas:

```
python lista1.extend(lista2) lista1 = lista1 + lista2
```

- **sort()**: Ordena la lista. Si tenemos una lista que mezcla distintos tipos, por ejemplo números y letras nos dará una excepción de tipo **TypeError**.

También podemos ordenar una lista con la función **sorted**:

```
sorted(list)
```

Recorrer una lista: enumerar

Uno de los usos más frecuentes de las listas es recorrer sus elementos. Ya dijimos que los bucles *for* están pensado para recorrer colecciones. Veamos cómo hacerlo con una lista:

```
amigos = ['Juan', 'Dionisio', 'Pepe', 'Manolo']
for amigo in amigos:
    print(f'Hola {amigo}')
```

A este proceso de recorrer una lista se le denomina enumerar o iterar. A veces necesitamos además de iterar el indicar la posición del elemento. Podemos hacerlo con la palabra reservada **enumerate**, obteniendo tanto el elemento como su posición:

```
amigos = ['Juan', 'Dionisio', 'Pepe', 'Manolo']
for posicion, amigo in enumerate(amigos):
    print(posicion, amigo)
```

Este ejemplo nos muestra que podemos utilizar varias variables en un mismo bucle *for*, algo que haremos con frecuencia.

Creación dinámica de listas

En ocasiones necesitamos generar una lista a partir de otra, realizando una operación sobre sus elementos. Podemos hacerlo con lo que ya sabemos, recorriendo nuestra lista y añadiendo los elementos a una nueva lista:

```
pi = 3.14
radios = [2.0, 35, 7.1]
circunferencia = [] # Creamos la lista vacía
for radio in radios:
    circunferencia.append(radio*pi)
```

Pero como ésto es algo muy frecuente, Python incorpora una manera muy cómoda de hacerlo, donde especificamos dentro de la lista la manera en la que calculamos cada elemento a partir de un bucle *for*:

```
pi = 3.14
radios = [2.0, 35, 7.1]
circunferencias = [pi * num for num in radios]
```

Matrices

Hemos dicho que una lista puede contener cualquier cosa. De hecho podemos hacer una lista que contenga listas, es lo que llamamos una matriz. Veamos un ejemplo

```
fila1 = [1, 3, 5]
fila2 = [9, 3, 5]
fila3 = [7, 3, 8]

matriz = [fila1, fila2, fila3]

print(matriz)

print(matriz[1][2])
```

Como vemos necesitamos 2 índices para acceder a un elemento dado.

Diccionarios

Un diccionario es una colección de pares etiqueta/valor, separados por el signo ":". También se usa la palabra clave o llave para referirse a la etiqueta.

Se usan las llaves, expresados de la siguiente forma (usaremos el que parece ser el ejemplo por excelencia de los diccionarios: los lenguajes de programación y su fecha de creación):

```
>>> lenguajes = { 'C':1972, 'python':1991, 'Java':1996}
>>> print(lenguajes)
{'C': 1972, 'python': 1991, 'Java': 1996}
```

Podemos crear un diccionario vacío con:

```
diccionario = {}
```

Para recuperar un valor preguntaremos por una llave:

```
>>> print(lenguajes['C'])
1972
```

Si el elemento no existe se produce una excepción de tipo **KeyError**.

Podemos evitar el error usando `get('clave')` que devolverá **None** si la clave no está en el diccionario. Podemos pasarle al método `get` un argumento que se devolverá como valor por defecto si la clave no existe:

```
>>> print(lenguajes.get('C++', 'Año desconocido'))
'Año desconocido'
```

Si queremos obtener un elemento y a la vez retirarlo del diccionario usaremos la función **pop()**.

Otra forma de ver si una clave está en el diccionario es usando **in**:

```
>>> 'BASIC' in lenguajes
False
```

Asignando un nuevo elemento se incluirá en el diccionario:

```
>>> lenguajes['C++'] = 1983
```

Podemos borrar un elemento usando la función **del**:

```
>>> del lenguajes['Java']
```

Podemos crear diccionarios con esta otra sintaxis:

```
>>> lenguajes = dict(C=1972, Java=1996, Python=1991)
>>> print(lenguajes)
{'C': 1972, 'Java': 1996, 'Python': 1991}
```

O a partir de listas, asignando a todos el mismo valor:

```
>>> oldLenguajes = ['Assembler', 'Lisp', 'Pascal']
lenguajesAntiguos = dict.fromkeys(oldLenguajes, 0)
>>> print(lenguajesAntiguos)
{'Assembler': 0, 'Lisp': 0, 'Pascal': 0}
```

Iterando en los diccionarios

Podemos iterar un diccionario usando las claves:

```
>>> for clave in lenguajes:
...     print(clave)
C
Java
Python
```

También podemos iterar por los valores usando la función **values()** (Cuidado con olvidar los paréntesis de **values()**, que es una función):

```
>>> for valor in lenguajes.values():
...     print(valor)
1972
1996
1991
```

Del mismo modo que hemos accedido a todos los valores almacenados en el diccionario, también podemos acceder a todas las claves usando la función **keys()**:

```
>>> for key in lenguajes.keys():
...     print(key)
C
Java
Python

>>> 'C++' in lenguajes.keys()

False
```

Aunque no es una lista, podemos usarla como tal.

Incluso podemos iterar por ambos a la vez usando la función *items()* (Cuidado con olvidar los paréntesis de *items()*, que es una función):

```
>>> for clave,valor in lenguajes.items():
...     print(clave, ' - > ',valor)
C - > 1972
python - > 1991
Java - > 1996
```

Cadenas y colecciones

Después de trabajar con listas podemos ver las cadenas como una lista de caracteres:

- Accedemos a cada carácter por índice *cadena[4]*.
- Podemos seleccionar una parte de una cadena *cadena[inicio:fin]*.

Veamos algunas funciones para trabajar con cadenas.

split: rompe una cadena en palabras

```
>>> frase = 'hola mundo'
>>> palabras = frase.split()
>>> print(palabras)
['hola', 'mundo']
```

Se le puede pasar como argumento el carácter separador, que por defecto es espacio.

join: genera cadena

Genera una cadena a partir de un separador y una lista

```
" ".join(['hola', 'mundo'])
```

Traducción de caracteres

Cuando trabajamos con algoritmos de encriptación sencillos podemos usar un diccionario para traducir el texto original en el texto codificado:

```
pt = dict()
pt['a'] = '7'
pt['e'] = ';'
pt['i'] = ' '
pt['o'] = '.'
pt['u'] = 'j'
# ....
```

A partir de un texto, generamos el texto codificado usando el diccionario que contiene la clave.

Conjuntos/Sets

También existen los **Conjuntos** (sets), que almacenan valores sin ordenar y que no pueden estar repetidos. Sólo se pueden guardar elementos inmutables (que no cambian) y que de los que se puede obtener un valor **hash** (algo similar a un id único).

La diferencia entre conjunto y lista es que en los conjuntos no se repiten elementos.

Además en los conjuntos no podemos acceder a los elementos individualmente.

Están pensados para trabajar las típicas operaciones entre conjuntos: uniones, intersecciones, diferencias,...

Los crearemos con los elementos entre llaves:

```
>>> s = {1, 2, 4}
```

Podemos añadir elementos con **add**

```
>>> print(s)
>>> s.add(5)
>>> print(s)
>>> s.add(5)
>>> print(s)
{1, 2, 4}
{1, 2, 4, 5}
{1, 2, 4, 5}
```

Y vemos que si intentamos añadir un elemento repetido, sólo aparecerá una vez.

Podemos hacer operaciones lógicas entre ellos, como por ejemplo esta operación **OR** en la que se muestra una unión de elementos:

```
>>> a = {1, 2, 3, 4}
>>> b = {3, 4, 5, 6}
>>> a | b
{1, 2, 3, 4, 5, 6}
```


Sería equivalente a una *unión*

```
>>> a.union(b)
{1, 2, 3, 4, 5, 6}
```

También podemos hacer una diferencia, que nos mostrará aquellos que están en uno y no el otro:

```
>>> a-b
{1, 2}
```

Zip

Al contrario de lo que pueda parecernos el nombre, la función **zip** no sirve para comprimir cosas sino para encajarlas. Quizás porque *zip* significa cremallera en inglés...

A partir de dos listas de elementos podemos generar una lista formada por pares de elementos, uno de cada lista

```
países = ['Francia', 'Alemania', 'España']
capitales = ['París', 'Berlín', 'Madrid']
```

```
for país, capital in zip(países, capitales):
    print(f'{país} capital {capital}')
```

También podemos generar un diccionario

```
diccionario = dict(zip(países, capitales))
print(diccionario)
```

Ejercicios colecciones

Vamos a ver algunos ejercicios para practicar lo aprendido con las colecciones.

Uniando 2 listas en un diccionario

Creemos un diccionario a partir de 2 listas: una contendrá las claves y otra los valores:

```
keys = ['Ten', 'Twenty', 'Thirty']
values = [10, 20, 30]

diccionario = {}
contador = 0
for key in keys:
    diccionario[key] = values[contador]
    contador += 1

print(diccionario)
```

Mezclar 2 diccionarios en uno

Unimos 2 diccionarios en uno que contendrá todos los pares:

```
dict1 = {'Ten': 10, 'Twenty': 20, 'Thirty': 30}
dict2 = {'Thirty': 30, 'Fourty': 40, 'Fifty': 50}

dict3 = dict1.copy() # Creamos un nuevo diccionario
dict3.update(dict2)
print(dict1)
```

Acceso a un elemento dentro de un diccionario

Se pueden crear estructuras anidadas dentro de un diccionario: el valor es un nuevo diccionario.

Acceder al valor del elemento 'history' en un diccionario que tiene esta estructura.

```
sampleDict = {
    "class":{
        "student":{
            "name":"Mike",
            "marks":{
                "physics":70,
                "history":80
            }
        }
    }
}

sampleDict["class"]["student"]["marks"]["history"]
```

Eliminar determinados pares de un diccionario

Vamos a ver cómo eliminar varios elementos de un diccionario:

```
diccionario = {
    "name": "Kelly",
    "age":25,
    "salary": 8000,
    "city": "New york"
}

clavesAQuitar = ["name", "salary"]
restantes = {}
for key in diccionario:
    if key not in clavesAQuitar:
        restantes[key] = diccionario[key]
print(restantes)
```

José Antonio Vacas Martínez

Aprende a programar Python



Logo Python

Versión 0.99.2



Licencia CC by SA by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Octubre 2022

Trabajando con ficheros

A lo largo de este tema vamos a aprender a trabajar con ficheros, lo que nos va a permitir almacenar información de manera permanente y recuperarla posteriormente.

Como práctica, haremos un programa que es capaz de contar las líneas, palabras y caracteres de todos los ficheros que tenemos en un directorio. También haremos que nuestro programa de las 20 Preguntas pueda guardar su conocimiento para partidas posteriores.

Administración de ficheros

En este capítulo vamos a aprender a gestionar ficheros, usando los módulos *os*, *pathlib* y *shutils* de Python..

Por gestionar entendemos el encontrarlos en las carpetas, ver los que tenemos disponibles, sus características y las operaciones más frecuentes: copiarlos, moverlos, borrarlos,...

Un módulo en Python es un conjunto de funciones, de código y de funcionalidad en general, que está orientado hacia satisfacer unas necesidades concretas. Por ejemplo existe un módulo *math*, que se encarga de incluir la funcionalidad para trabajar las funciones matemáticas u otro que se llama *pyGame*, que está pensado para ayudar a programar videojuegos.

En un tema posterior veremos en detalles el uso de módulos y cómo utilizarlos. No obstante, vamos a empezar a utilizarlos ya en este, usando los módulos relacionados con ficheros.

Por ahora es suficiente con saber que para trabajar con un módulo sólo tenemos que poner al principio del código la palabra reservada **import** y el nombre del módulo. De esta manera, antes de ejecutar nuestro código, se cargará toda esa funcionalidad contenida en el módulo y la tendremos disponible para utilizarla.

Cuando queramos utilizar una funcionalidad de un módulo concreto, pondremos el nombre del módulo, un punto, y el nombre de esa funcionalidad que vamos a utilizar.

Por ejemplo para usar la función *listdir* del usando módulo *os*, escribiremos *os.listdir()*.

Existen varios módulos que nos permiten trabajar con ficheros. De hecho bastante funcionalidad está replicada en varios de ellos, usaremos uno u otro dependiendo de lo que vayamos a hacer finalmente con los ficheros:

- Si vamos a trabajar solamente revisando dónde están los ficheros pero no vamos a leer su contenido el módulo recomendado es `os`.
- Pero si además de manipularlo lo que queremos es leer o escribir su contenido se recomienda usar el módulo `pathlib`

Operaciones con ficheros

Vamos a ver algunas de operaciones más frecuentes

- Podemos crear directorios con `os.mkdir(directorio)`:

```
import os
os.mkdir('dir') ## Creamos el directorio 'dir'
```

- También podemos crear varios niveles con `os.makedirs(dir1/dir2)`:

```
import os
os.makedirs('nivel1/nivel2/nivel3')
```

- Para borrar un fichero usaremos `remove(fichero)`:

```
import os
os.remove('data.txt') ## Borramos un fichero
```

- Para mover ficheros entre directorios, sólo tenemos que cambiar el nombre usando `rename`. Por ejemplos tenemos varios ficheros en el directorio raíz y queremos moverlos a un directorio 'prueba':

```
import os
os.mkdir('prueba') # crea el directorio '1'
os.rename('irpf_app.py', 'prueba/irpf_app.py') # mueve el fichero dentro del directorio
```

- Si queremos borrar un directorio usaremos `rmdir(dir)`:

```
import os
os.rmdir('1') # borra el directorio
```

- Podemos saber el directorio actual con `os.getcwd()`:

```
import os
directorioActual = os.getcwd()
print(f'Estamos en el directorio {directorioActual})
```

Dentro del módulo `os` existe un grupo de funcionalidad que está pensado para trabajar específicamente con el "path".

Dado que python está pensado para trabajar con diferentes sistemas operativos y algunos de ellos utilizan distinto separador, podemos garantizar que usamos el correcto con `os.path.sep`.

Podemos crear una función "mover", que mueva un fichero creando directorio si es necesario:

```
import os
```

```
def mover(fichero, nuevoDirectorio):
    """
    función que hace un move de un fichero
    """
    if nuevoDirectorio not in os.listdir(): # Si no está creado lo creamos
        os.mkdir(nuevoDirectorio)
    if nuevoDirectorio[-1] != os.path.sep: # añadimos la barra
        nuevoDirectorio += os.path.sep
    os.rename(fichero, nuevoDirectorio+fichero)
```

Cuando intentamos hacer estas operaciones puede ocurrir que no tengamos permiso para hacerla, bien por limitaciones de los privilegios del usuario o por los permisos de las carpetas. En ese caso se produce una excepción `PermissionError`

Recorriendo directorios y sus contenidos

Podemos recuperar los ficheros y directorios de un path concreto usando la función `listdir('path')` que nos devuelve una lista con todo el contenido de ese path:

```
import os
for fichero in os.listdir('directorio'):
    print(fichero)
```

A partir del nombre del fichero o directorio podemos recuperar el path absoluto con `os.path.abspath(path)`:

```
import os
for fichero in os.listdir('directorio'):
    print(f'{fichero} -> {os.path.abspath(fichero)}')
```

En el caso contrario, podemos recuperar el nombre del fichero y la carpeta que lo contiene usando `os.path.split(path)`:

```
import os
>>> os.path.abspath('main.py')
'/home/javacasm/Descargas/main.py'
>>> os.path.split(os.path.abspath('main.py'))
('/home/javacasm/Descargas', 'main.py')
```

Podemos recuperar el nombre del fichero de un path con `os.path.basename(path)` y el directorio que lo contiene con `os.path.dirname(path)`:

```
>>> os.path.basename('/home/javacasm/Descargas/main.py')
'main.py'
>>> os.path.dirname('/home/javacasm/Descargas/main.py')
'/home/javacasm/Descargas'
```

En el caso de que no podamos acceder al fichero se producirá una excepción de tipo `FileNotFoundError`.

Información sobre ficheros y directorios

En muchas ocasiones queremos saber si un fichero o directorio existe. Podemos saberlo con `os.path.exists(path)`, que nos devolverá `True` si existe y `False` en caso contrario.

También podemos preguntar si es un fichero o directorio con `os.path.isfile(f)` o `os.path.isdir(f)`:

```
import os
for f in os.listdir():
    if os.path.isfile(f):
        print(f'"{f}" es fichero')
    else :
        print(f'"{f}" es un directorio')
```

Vamos a ver ahora cómo obtener información (por ejemplo el tamaño y las fechas de modificación) sobre un fichero concreto, lo que nos va a mostrar una vez más que en python casi todo se puede hacer de varias formas:

1. Usando el módulo `os.path`

```
import os
import time

nombreFichero = 'fichero.txt'
print('Información del fichero ' + nombreFichero)
print('Tamaño: ',os.path.getsize(nombreFichero))
print('Fecha de creación: ',time.ctime(os.path.getctime(nombreFichero)))
print('Fecha          de          última          modificación: ',time.ctime(os.path.getmtime(nombreFichero)))
print('Fecha de último acceso: ',time.ctime(os.path.getatime(nombreFichero)))
```

Donde hemos usado la función `time.ctime` para convertir el formato de fecha Epoch (segundos desde el 1 de enero de 1970) que se usa en el sistema a un formato más legible.

2. Usando `os.stat()`: Si vamos a recuperar muchos detalles es más eficiente usar la función `stat()` que nos devuelve una estructura con todos los datos:

```
>>> info = os.stat('fichero.txt')
>>> info
os.stat_result(st_mode=33204, st_ino=2345119, st_dev=2051, st_nlink=1,
st_uid=1000, st_gid=1000, st_size=118, st_atime=1612791889,
st_mtime=1577972527, st_ctime=1577972530)
>>> time.ctime(info.st_atime)
'Mon Feb  8 14:44:49 2021'
```

En el caso de que no podamos acceder al fichero se producirá una excepción de tipo `FileNotFoundError`.

Usando `pathlib`

En versiones más recientes de Python, las funciones de manejo de ficheros se han replicado en el módulo `pathlib`, estando casi todas las funciones que hemos comentado en `pathlib.path`. Además de gestionar los ficheros se ha incluido alguna funcionalidad para leerlos o escribir en ellos. Es de suponer que en futuras versiones esta funcionalidad se migre completamente del módulo `os`.

Veamos alguna utilidad de `pathlib`, como es el encontrar los ficheros cuyo nombre cumple un patrón, para lo que podemos usar la función `glob(patron)`:

```
import pathlib

ficherosCodigo = pathlib.Path('.').glob('*.py')
for f in ficherosCodigo:
    print(f'fichero:{f}')
    print(f'nombre:{f.stem}')
    print(f'extensión:{f.suffix}')
    print(f'tamaño:{f.stat().st_size}')
```

Del mismo modo que en el módulo `os` podemos preguntar si existe con `exist()` o si es un fichero con `is_file()` o si es un directorio con `is_dir()`.

Copiar ficheros

Si lo que queremos hacer es copiar ficheros usaremos el módulo `shutils` y sus funciones `copy` o `copy2` según queramos que se copie sólo el fichero o también sus metadatos.

También podemos copiar toda una rama de directorios con sus contenido con `copytree`.

Este módulo también nos permite mover ficheros con `move` y borrar directorios y su contenido con `rmtree`.

Todos estos métodos necesitan que les proporcionemos ficheros o directorio origen y destino.

Exploración recursiva de directorios

Con lo aprendido vamos a hacer un programa que navegue recursivamente un directorio mostrando información del ficheros que encuentre y entrando en todos los subdirectorios para explorarlo

```
import os

listaDirectorios = ['./Descargas']

def showInfo(fichero):
    info = os.stat(fichero)
    print(f'fichero: {fichero} tamaño: {info.st_size}')

def exploraDir(directorio):
    global listaDirectorios
    print('Explorando ' + directorio)
    for fichero in os.listdir(directorio):
        if os.path.isdir(fichero):
            listaDirectorios.append(fichero)
        if os.path.isfile(fichero):
            showInfo(fichero)

while len(listaDirectorios) > 0:
    directorio = listaDirectorios.pop()
    exploraDir(directorio)
```

Lectura de ficheros

Vamos a aprender a leer el contenido de ficheros. Para ello tenemos que empezar abriendo el acceso al fichero con la función `open` a la que pasamos como argumentos el **nombre del fichero** y el **modo** en que queremos abrir el fichero.

Veamos los distintos modos de lectura:

- "r" para lectura.
- "t" para lectura de texto, podremos recuperar línea a línea.
- "b" para lectura de datos binarios.

Por defecto se sobreentiende el modo "rt".

La función `open` nos devuelve una referencia al fichero que usaremos para acceder al mismo

```
f = open('fichero.txt', 'rt')
for linea in f.readlines():
    print(linea)
f.close()
```

Cuando terminemos de usarlo llamaremos a la función `close()` para cerrarlo y liberar recursos.

En el ejemplo anterior veremos que se muestran las líneas separadas en 2 líneas, lo que es debido a que también se lee el carácter de salto de línea `"\n"` que hay en el fichero y se imprime además el salto de línea de `print`. Podemos evitarlo poniendo el argumento de `print` `end=""` para evitar el salto de línea extra.

Hemos leído todas las líneas del fichero con `readlines()` pero también podíamos haber leído una a una con `readline()` incluso carácter a carácter con `read()`, de N en N con `read(N)`.

En el caso de que no se pueda abrir el fichero se produce una excepción de tipo `FileNotFoundException`.

Codificación de ficheros de texto

Existen diferentes formas de codificar los caracteres. En el inicio de los tiempos de la informática se usaba la codificación **ASCII**, que utilizaba sólo 7 bits, con lo que no permitía codificar más de 128 caracteres ($2^7 = 128$), lo cual era suficiente para trabajar con el idioma inglés, pero dejaba fuera acentos y caracteres de otros lenguajes, por ello se amplió a 8 bits, apareciendo diversas versiones para distintos idiomas. El **ISO-8859-1** o **Latín 1** era la versión correspondiente al español, solucionaba parcialmente el problema porque se usaba una codificación distinta para cada idioma.

Para resolver este problema el **Unicode** que utiliza 32 bits donde ya cabían todos los caracteres de todos los idiomas permitiendo usar 2^{32} (más de 4000 millones) de posibles caracteres.

Para optimizar su uso se creó **Utf-8** (que permite utilizar un tamaño variable de los caracteres y ahorra espacio)

Python 3.x usa Unicode para las cadenas de caracteres, pero cuando abrimos un fichero, para la lectura se usa por defecto la codificación del sistema operativo. Por

todo ello cuando abrimos un fichero de texto y para evitar errores podemos añadir un parámetro de codificación, obligando a que se use una codificación determinada.

Para asegurarnos que se abre con codificación Utf-8 (la más estándar hoy día) añadimos **encoding = "utf8"**

```
...  
file = open(nombre_fichero, "rt", encoding="utf8")  
...
```

Si intentamos leer un fichero con una codificación diferente podemos obtener un error de *"UnicodeDecodeError: 'charmap' codec can't decode byte 0x81 in position 74: character maps to "*, por ejemplo si trabajamos en Windows que usa la codificación **cp1252**.

Escritura de ficheros

Para escribir datos en un fichero, lo abrimos en el modo "w" de escritura.

```
import os  
f = open("myfile.txt", "wt"):  
f.write("Hello world!")  
f.close()  
print(os.listdir()) # comprobamos que se ha creado el fichero
```

El contenido que se escribe es literalmente el que nosotros ponemos explícitamente, con lo que si queremos que se añada un final de línea tendríamos que añadirlo manualmente.

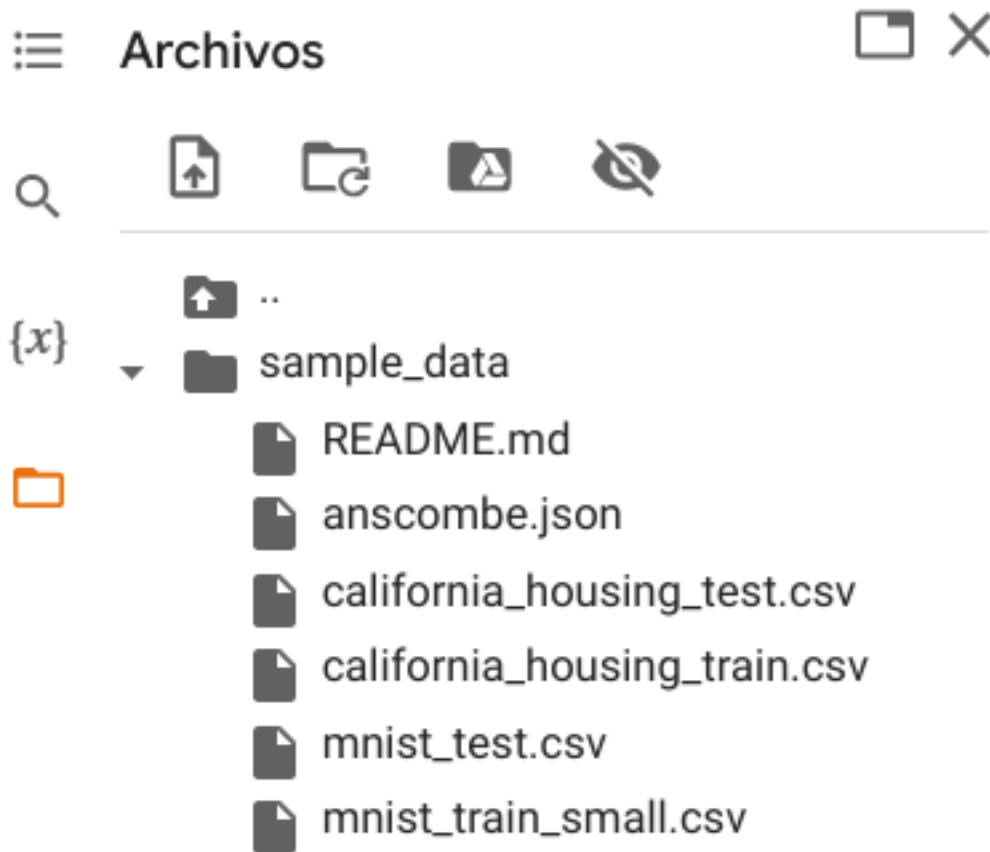
En el caso de que el fichero ya existiera usando el modo "w" éste se sobrescribe. También podemos añadir al contenido al final usando el modo "a".

Si no podemos escribir por permisos o por otra razón se producirá una excepción del tipo *PermissionError*.

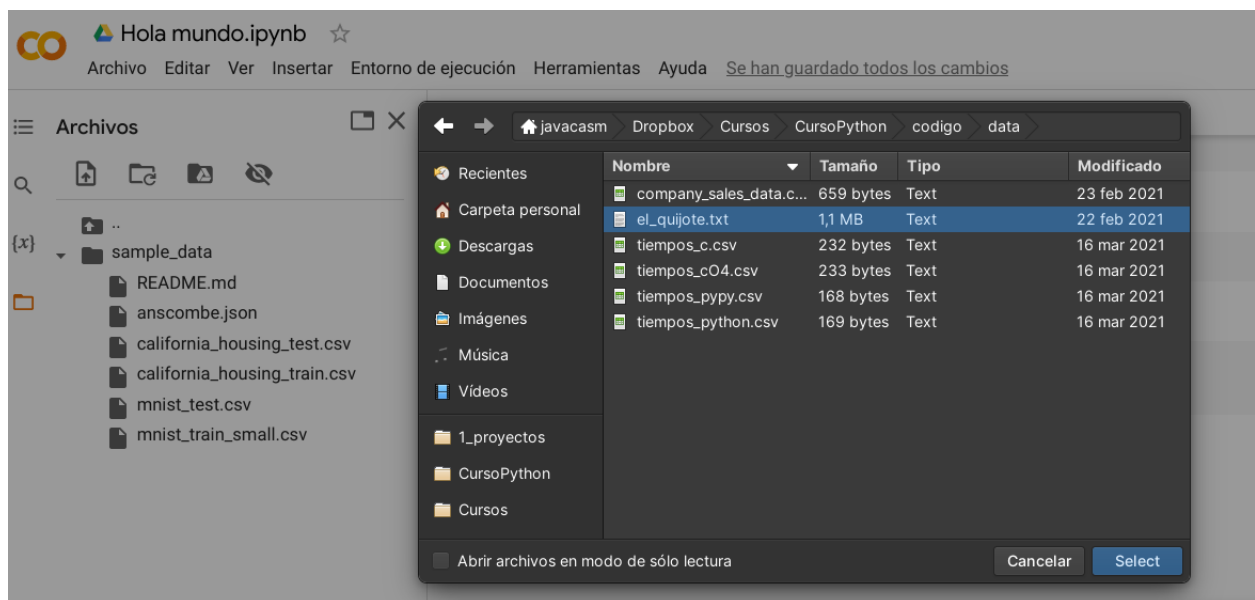
Gestión de ficheros en Google Colab

Vamos a ver cómo podemos incorporar ficheros a nuestro proyecto.

Tenemos un gestor de ficheros sin más que pulsar el icono de "Carpeta" de la barra lateral. Ahí podremos ver las carpetas y ficheros. Por defecto incluye una carpeta de ejemplos (sample_data). Desde ahí podemos añadir, borrar, renombrar, ... nuestros ficheros

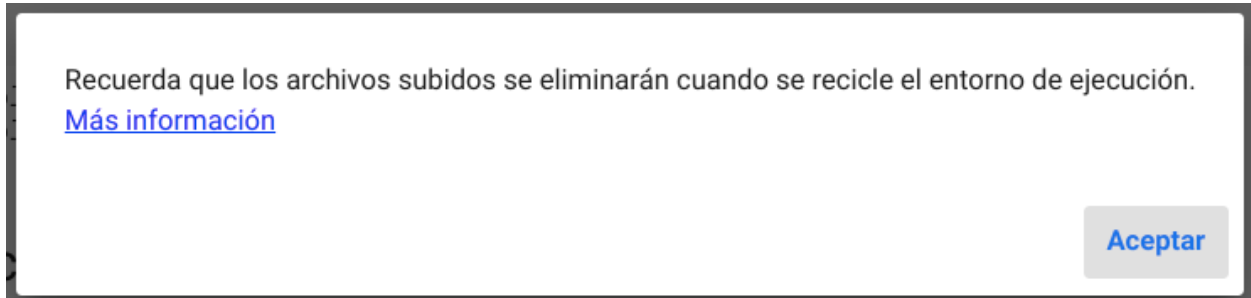


Podemos subir los ficheros desde nuestro ordenador, pulsando el primero icono



También podemos descargar cualquier fichero a nuestro ordenador.

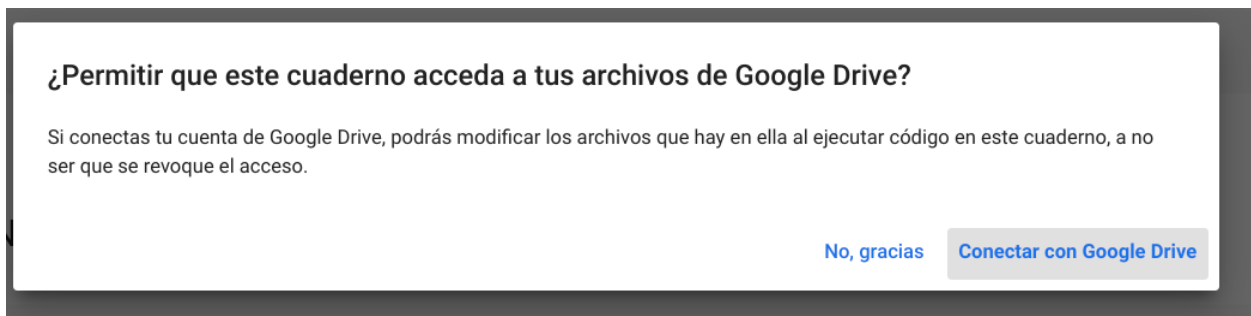
Hay que recalcar que **los ficheros que añadimos a nuestra sesión se pierden cada vez que desconectamos del entorno de ejecución** o lo reciclamos.



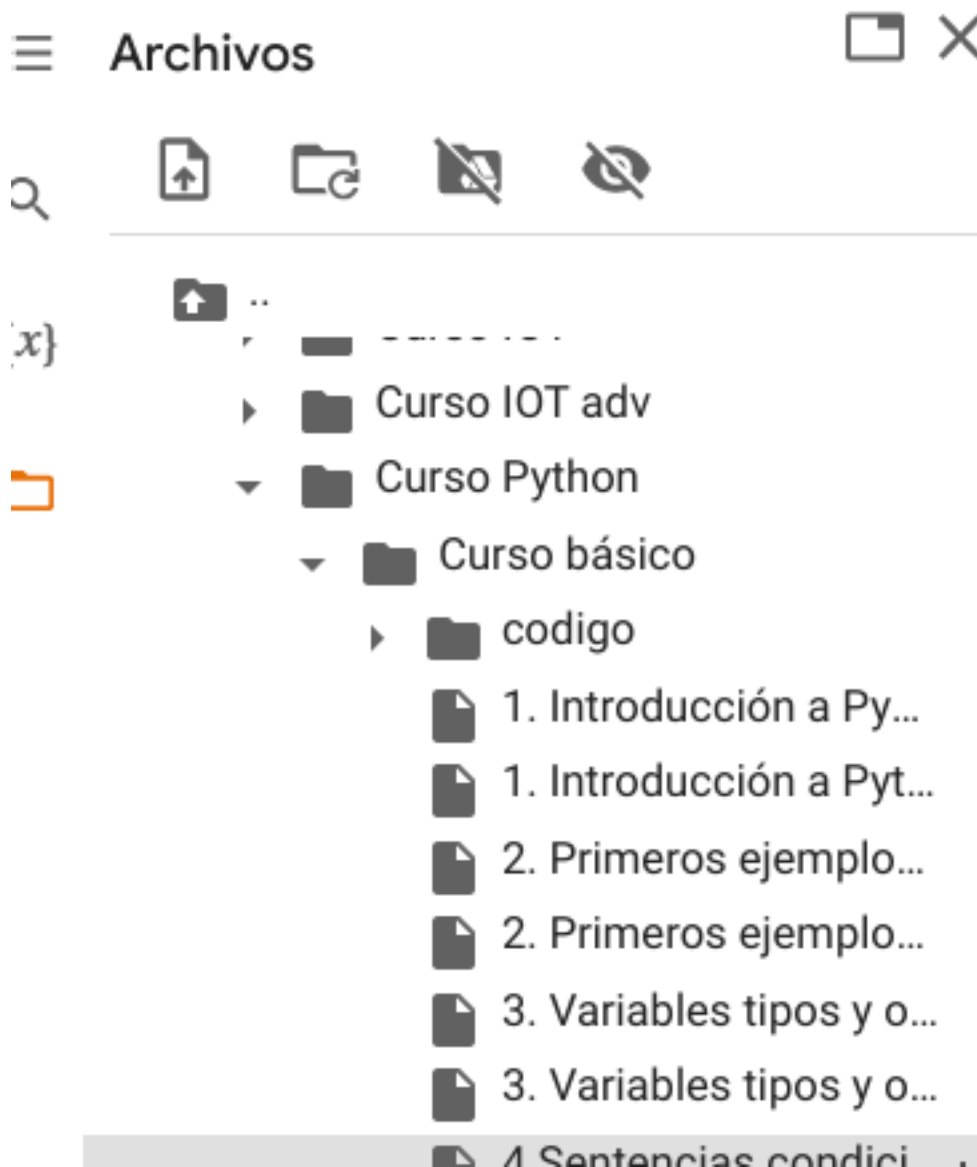
Por esto vamos a ver formas de acceder a documentos en otros almacenamientos como Google Drive o Github.

Conexión con Google Drive

Podemos conectar con Google Drive para acceder a los ficheros allí alojados. Al pulsar sobre el icono de "Google Drive" seleccionaremos nuestra cuenta y nos pedirá permiso para acceder a los ficheros:



Cuando conectamos con Google Drive nos aparecerá una carpeta "drive" entre nuestros ficheros y ahí se mostrarán los ficheros de Drive. Este paso puede tardar si tenemos muchos ficheros:

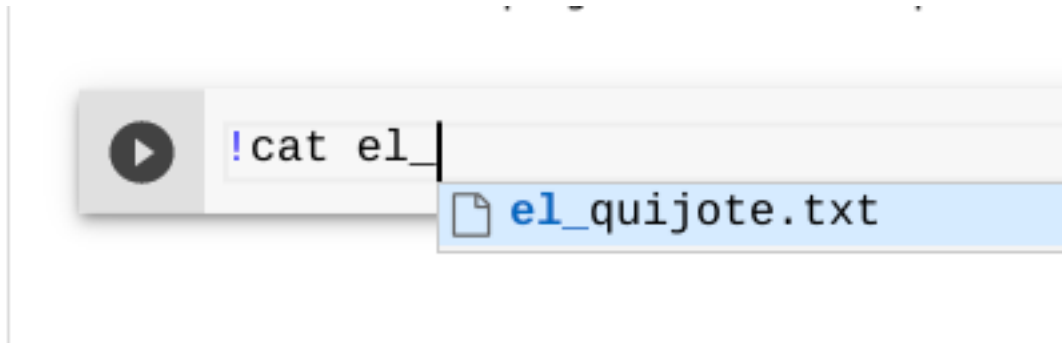


Tras conectar (“montar” en palabras técnicas) con Drive podemos desconectar (desmontar) pulsando en la opción correspondiente.

Comando directos

Podemos ejecutar comandos sobre el servidor al que estamos conectados como haríamos sobre cualquier máquina linux. Para ello antepone una “!” (admiración) al comando por ejemplo para trabajar con ficheros podemos usar:

- **!ls** para listar los ficheros del sistema
- **!cp** para copiar un fichero, por ejemplo “!cp eL_quijote.txt libro1.txt”



- **!rm fichero.txt** para borrar "fichero.txt"
- **!cat fichero.txt** para mostrar el contenido de "fichero.txt"
- **!ps -ef** nos muestra los procesos que se están ejecutando

Conexión con Github

Podemos interaccionar con un repositorio github usando el comando **git clone** como lo haríamos en nuestro ordenador.

- **!git clone repositorio.git**



Aprende a programar Python



Logo Python

Versión 0.99.2



Licencia CC by SA by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Octubre 2022

Módulos y paquetes

Como hemos visto, un módulo o paquete no es otra cosa que una colección de código, clases, variables y funciones que podemos utilizar desde nuestros programas. Lo importamos al principio del programa y a partir de ahí podremos utilizarlo en nuestro código.

Por **módulo** entendemos un fichero que incluye código y un **paquete** será un conjunto de módulos que se suelen incluir en un directorio.

```
paquete
├── modulo1.py
├── modulo2.py
└── modulo3.py
```

Y dentro de cada fichero moduloX.py tendríamos funciones, clases, ...

Existen diferentes formas de importación según queramos importar todo el paquete o solamente unos módulos del mismo.

Con “*import paquete*” lo importamos completo. Para usarlo usaremos sus clases, funciones o variables precedidas del nombre del módulo, al que llamaremos *espacio de nombres* (namespace):

```
import os

ficheros = os.listdir('/tmp')
os.rename('fichero1', 'fichero2')
```

En nuestro “paquete” imaginario sería:

```
import paquete

paquete.modulo1.funcion1()
```

El código dentro del módulo se agrupa según el espacio de nombres, por ejemplo *os.path.isfile()*.

Si queremos importar sólo alguna parte del código podemos usar la estructura de “*from ... import clase*”, pudiendo evitar la referencia al módulo. Por ejemplo:

```
from os import path
path.isfile('test')
```

Para agilizar el uso del código de los módulos podemos definir un alias con la palabra reservada “as” en cualquier de los formatos (*import* o *from ... import*)

```
import os.path as p
p.isfile('test')
```

En nuestro ejemplo, si sólo queremos usar el módulo2 haríamos

```
from paquete import modulo2
```

...

¿Dónde se buscan los módulos?

Cuando importamos un paquete ¿dónde se busca este paquete?:

1. En primer lugar se busca en la librería estándar de python, es decir entre todos los paquetes que incluye la versión que estamos usando
2. Después se busca en todos los directorios incluidos en “sys.path”, donde se incluyen en este orden:
 - a. El directorio actual donde estamos ejecutando nuestro código
 - b. Los directorios de instalación de python y algunos otros según un orden concreto.

Podemos ver lo que incluye nuestro `sys.path` con este sencillo código:

```
>>> import sys
>>> for p in sys.path:
...     print(p)
...
```

```
/usr/lib/python36.zip
/usr/lib/python3.6
/usr/lib/python3.6/lib-dynload
/home/javacasm/.local/lib/python3.6/site-packages
/home/javacasm/Dropbox/Cursos/Drones/DJITelloPy
/usr/local/lib/python3.6/dist-packages
/usr/lib/python3/dist-packages
```

El estar ordenado de esta manera hace que podamos incluir un paquete que sobrescriba otro. Por ejemplo poniendo un paquete “os” en el directorio actual, al hacer “import os” se importaría el nuestro.

Importando nuestro código

Cuando creamos un programa cuyo código está distribuido entre varios ficheros, podemos importar la funcionalidad como si un módulo se tratara, haciendo simplemente `import fichero` (sin la extensión .py)

Accederemos a la funcionalidad con `fichero.funcion` del mismo modo.

Puedes probarlo importando el código del fichero de las 20 preguntas y usándolo desde tu propio programa.

Cuando hacemos esto tenemos que distinguir si estamos usando el código como un módulo o como un programa principal.

En la siguiente sección vamos a ver cómo hacerlo.

Propiedades de los módulos

Cuando ejecutamos un intérprete Python existen una [serie de variables](#) con valores características, como por ejemplo:

- **name** que se refiere al nombre completo del módulo en ejecución. Cuando ejecutamos un fichero, el valor de este atributo será **'main'**, cuando importamos un módulo será el nombre del módulo.

Podemos aprovechar esto para saber si nuestro código se está ejecutando directamente o a través de una importación.

```
if __name__ == '__main__':
    myMain()
else:
    print('Me han importado')
```

- **version** nos dice la versión de un módulo usando

```
import os
os.__version__
```

- **file** nombre del fichero

Vamos a probarlo. Creamos un fichero *Prop_modulos.py* con este contenido:

```
'''
Módulo de prueba
'''

try:
    print(f'__name__: {__name__}')
    print(f'__file__: {__file__}')
    print(f'__loader__: {__loader__}')
    print(f'__package__: {__package__}')
    print(f'__path__: {__path__}')
except Exception as e:
    print(f'Error: {e}')
```

Si lo ejecutamos con *python3 Prop_modulos* obtenemos:

```
__name__: __main__
__file__: Prop_modulos.py
__loader__: <_frozen_importlib_external.SourceFileLoader object at
0x7fcb9d9d02b0>
__package__: None
Error: name '__path__' is not defined
```

En cambio si desde un intérprete python3 hacemos *import Prop_modulos* veremos:

```
>>> import Prop_modulos
__name__: Prop_modulos
__file__: /home/javacasm/Dropbox/Cursos/CursoPython/codigo/Prop_modulos.py
__loader__: <_frozen_importlib_external.SourceFileLoader object at
0x7fd7a67433c8>
__package__:
Error: name '__path__' is not defined
```

Si ahora hacemos *help(Prop_modulos)* veremos que nos muestra parte de esta información.

Librería Estándar

Actualmente Python incorpora una enorme librería por defecto, es la llamada Librería Estándar y podemos ver todos los detalles en su [documentación](#)

Ya conocemos algunos de estos módulos como *os*, *time* o *random*. Vamos a ver una descripción de los principales módulos:

- **os** todo lo relacionado con el sistema operativo (en general): ficheros, hilos, variables del entorno, procesos, permisos, usuarios, redes,
- **sys** temas específicos al sistema operativo donde se está ejecutando: propiedades de los ejecutables, de la configuración del sistema,...
- **csv** para trabajar en un nivel básico con archivos de datos de tipo csv
- **math** todo lo relacionado con las matemáticas: funciones, constantes,...
- **glob**: para trabajar con comodines al listar ficheros.
- **datetime**: fechas y horas
- **statistics**: para realizar operaciones estadísticas.
- **zip**: para trabajar con ficheros comprimidos zip.

- **time**: medida del tiempo y esperas.
- **Multimedia**: para reproducir y generar ficheros multimedia
- **Protocolos de Internet**: para utilizar los protocolos más frecuentes de internet.
- **random**: números y elecciones aleatorios

Ya hemos trabajado algunos de ellos y ahora vamos a ver ejemplos de otros.

Módulo sys

El módulo **sys** tiene las utilidades propias del sistema operativo donde estamos ejecutando el intérprete:

- `sys.argv`: argumentos con los que hemos ejecutado el programa
- `sys.executable`: ejecutable del intérprete actual.
- `sys.exit()`: sale del programa python actual.
- `sys.float_info`: información sobre las características del tipo decimal.
- `sys.getsizeof(objeto)`: nos dice la memoria que ocupa el objeto indicado.
- `sys.maxsize`: máximo entero que podemos usar.
- `sys.modules`: diccionario con los módulos cargados.
- `sys.version`: versión del intérprete que estamos usando.

Módulo math

En el módulo **math** como no podía ser de otra forma nos vamos a encontrar todo tipo de funciones matemáticas:

- Funciones trigonométricas, como *sin*, *cos*, *tan*,... y sus inversas *asin*, *acos*, *atan*, ...
- Raíces cuadradas con *sqrt*
- Exponenciales con *exp* y logaritmos con *log*
- Constantes como *math.pi*, *math.e*, *math.inf* (infinito)

Como ejemplo vamos a ver las diferentes formas de redondear un número decimal a entero

Empezamos por la función *round()* que produce el resultado al que estamos acostumbrados:

- Los números con parte decimal igual o menor de 0.5 se redondean al entero anterior

- Si la parte decimal es mayor que 0.5 se redondea al entero siguiente:

```
>>> numeros = (0.5, 0.51, 1.6, 2.5, 2.51)
>>> print('Round')
>>> for numero in numeros:
...     print(f'{numero} -> {round(numero)}')
Round
0.5 -> 0
0.51 -> 1
1.6 -> 2
2.5 -> 2
2.51 -> 3
```

El módulo *math* incluye otras 2 formas de redondear:

- *math.floor()* que redondea siempre al entero anterior

```
>>> import math
>>> print('math.floor')
>>> for numero in numeros:
...     print(f'{numero} -> {math.floor(numero)}')
math.floor
0.5 -> 0
0.51 -> 0
1.6 -> 1
2.5 -> 2
2.51 -> 2
```

- *math.ceil()* redondea al entero superior ``

```
>>> import math
>>> print('math.ceil')
>>> for numero in numeros:
...     print(f'{numero} -> {math.ceil(numero)}')
math.ceil
0.5 -> 1
0.51 -> 1
1.6 -> 2
2.5 -> 3
2.51 -> 3
```

Trabajando con fracciones

Aunque no forman parte del módulo *math*, sino que tienen su propio módulo, en la biblioteca estándar de Python podemos encontrar las clases necesarias para trabajar con *fracciones*. El módulo no podía tener otro nombre que **fractions**, donde trabajaremos con la clase **Fraction**, definiendo su *numerador* y su *divisor*:

```
>>> from fractions import Fraction
>>> f1 = Fraction(1,2)
>>> f2 = Fraction(1,3)
```

Por supuesto podemos imprimirlas:

```
>>> print(f1)
1/2
>>> print(f2)
1/3
```

Vemos que podemos operar entre ellas si recurrir a un valor decimal, lo que las hace más exactas:

```
>>> f3 = f1 + f2
>>> f3
Fraction(5, 6)
```

Y también podemos hacer operaciones entre fracciones y números “normales”:

```
>>> f4 = 3 * f2
>>> f4
Fraction(1, 1)
>>> 1 == f4
True
```

y que en todo momento se simplifican los resultados:

```
>>> f5 = Fraction(3,24)
>>> f5
Fraction(1, 8)
```

Además el módulo tiene otros métodos, como por ejemplo **gcd** que nos permite calcular el máximo común divisor:

```
>>> import fractions
>>> fractions.gcd(15,5)
5
```

Ficheros comprimidos

Existen varios módulos para trabajar con [tareas propias de compresión y descompresión](#): `zlib`, `gzip`, `bz2`, `lzma`, `tarfile` y `zipfile`.

Vamos a ver un ejemplo de cómo trabajar con un fichero zip estándar. El uso es similar al tratamiento de los ficheros normales:

- Lo abrimos indicando si lo queremos para lectura “r” o para escritura “w”
- ```
from zipfile import ZipFile
```

```
ficheroZip = ZipFile('ficheroZIP.zip', mode='r') # Lo abrimos en modo lectura
"r"
```

```
print(ficheroZip.namelist()) # mostramos los ficheros que incluyen
ficheroZip.extractall() # extraemos todos los ficheros
```

```
ficheroZip.close() # Lo cerramos
```

Si lo que queremos es crear un fichero zip con varios ficheros incluidos haremos lo siguiente:

```
from zipfile import ZipFile
```

```
ficheroZip = ZipFile('nuevoFicheroZIP.zip', mode='r') # Lo abrimos en modo
escritura "w"
```

```
ficheroZip.write('fichero1.txt') # añadimos los ficheros
ficheroZip.write('fichero2.txt')
ficheroZip.write('fichero3.txt')
```

```
ficheroZip.close() # Lo cerramos
```

## Aleatoriedad con random

Podemos usar el módulo `random` cuando necesitamos algo relacionado con lo aleatorio .

Tenemos varias posibilidades dependiendo de lo que necesitemos: \*  
`randint(minimo,maximo)`: Nos da un número entero comprendido entre el valor mínimo y máximo incluyendo los dos extremos:

```
from random import randint
Recordar que se incluye el máximo
minimo <= randint(minimo,maximo) <= maximo
for i in range(10):
 print(randint(0,10))
```

- `choice(coleccion)` : Devuelve uno de los elementos de la colección que le hemos pasado:

```
from random import choice
colores = ('rojo', 'verde', 'azul', 'blanco', 'negro', 'amarillo')
for i in range(5):
 print (choice(colores))
```

- `sample(coleccion, N)`: Elige N elementos distintos de la colección que la hemos pasado y los devuelve como una tupla

```
from random import sample
colores = ('rojo', 'verde', 'azul', 'blanco', 'negro', 'amarillo')
print(sample(colores,4))
```



- `shuffle(lista)`: desordena aleatoriamente la lista que le pasamos.

Ejemplo: Generador de password

```
chars = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ.,-:;<>][_+ '
longitud = int(input('¿longitud de la contraseña? '))
for i in range(longitud):
 passwd+=random.choice(chars)
print(passwd)
```

A veces, en contra de lo que pudiera parecer, queremos que se reproduzca una misma secuencia de números aleatorios (para pruebas, para cierta predictibilidad,...). En ese caso podemos hacer usar el método `seed(semilla)` que hará que toda la secuencia se repita si usamos la misma *semilla*:

```
>>> from random import seed, randint
>>> seed(666)
>>> for i in range(5):
... print(randint(0,10))
7
6
6
4
8
>>> seed(666)
>>> for i in range(5):
... print(randint(0,10))
7
6
6
4
8
```

## Midiendo tiempos

El módulo `time` está pensado para medir tiempos.

```
>>> import time
>>> print(time.time()) # segundos desde 1/1/1970
1664454320.520648
>>> print(time.localtime()) # estructura con los datos de fecha y hora
time.struct_time(tm_year=2022, tm_mon=9, tm_mday=29, tm_hour=14, tm_min=25,
tm_sec=49, tm_wday=3, tm_yday=272, tm_isdst=1)
>>> print(time.strftime('%Y-%m-%d_%H:%M:%S',time.localtime()))
2022-09-29_14:28:06
```

- `time()`: Nos da un número decimal donde la parte entera son los segundos desde el 1 de enero de 1970 y la decimal la fracción de segundo. Este formato se conoce como **timestamp**

- `localtime()` Nos da una estructura con la fecha y hora descompuesta
- `strftime(formato)`: nos devuelve una cadena con el dato de fecha y hora con el formato que le indicamos
- `sleep(segundos)`: detiene la ejecución del programa durante los segundos indicados. Este valor puede ser decimal.

También podemos usar `time()` para ver el tiempo que ha pasado entre dos momentos:

```
>>> inicio = time.time()
>>> final = time.time()
>>> print(f'Han pasado {final-inicio} segundos')
Han pasado 8.950223922729492 segundos
```

## Operando con fechas con datetime

El módulo `datetime` está pensado para operar con fechas.

- `datetime(año,mes,día, hora, minuto, segundo)` crea un objeto con fecha (podemos omitir los que valores de hora, minuto y segundo).
- `datetime.year` o `datetime.month`, ... nos da los valores esperados.
- `datetime.now()` nos da la fecha actual
- `datetime.strftime(formato)` convierte la fecha a una cadena con el formato indicado:

```
>>> from datetime import datetime
>>> datetime.now().strftime('%Y-%m-%d_%H:%M:%S')
'2021-03-25_20:52:38'
```

- Podemos convertir un valor de tipo `time` (timestamp) a `datetime` con:

```
>>> datetime.fromtimestamp(time.time())
datetime.datetime(2022, 9, 29, 14, 31, 43, 116377)
```

- Podemos restar fechas, obteniendo una estructura `timedelta` de resultado, de donde podemos extraer los días con `days`, los segundos con `seconds` y los microsegundos con `microseconds`.

### Código

```
>>> from datetime import datetime
>>> nacimiento = datetime(1970,12,25,15,0,0) # Fecha y hora de nacimiento
>>> hoy = datetime.now()
>>> diferencia = hoy - nacimiento
>>> print(f'han pasado {diferencia}')
han pasado 18905 days, 23:09:40.741071
>>> print(f'tienes {diferencia.days} días, y {diferencia.seconds} segundos')
tienes 18905 días, y 83380 segundos
>>> horas = diferencia.seconds // 3600 # segundos que tiene 1 hora
```

```
>>> restoSegundos = diferencia.seconds - horas * 3600
>>> minutos = restoSegundos // 60
>>> segundos = restoSegundos - minutos * 60
>>> print(f'o {diferencia.days} días, {horas} horas, {minutos} minutos y {segundos} segundos')
o 18905 días, 23 horas, 9 minutos y 40 segundos
```

- Para obtener el día actual podemos usar `today()`

```
from datetime import date
hoy = date.today()
print(hoy)
para formatearlo a Mes día, año
print(hoy.strftime('%B %d, %Y'))
```

```
para saber el día de la semana
print(hoy.weekday())
```

Vemos que nos muestra el número de día de la semana.

También podemos trabajar con los nombres de los días de la semana o de los meses, para lo que usaremos el módulo **calendar**, que dispone de varias listas con nombres:

- **month\_name** con los nombres de los meses
- **month\_abbr** con las abreviaturas de los nombres de los meses
- **day\_name** con los nombres de los días
- **day\_abbr** con las abreviaturas de los nombres de los días

La lista de los meses tiene un elemento vacío en la primera posición (posición 0) para que se correspondan los nombres con los números habituales (enero - 1, ....), con lo que podemos sacar una lista de los nombres de los meses con:

```
>>> import calendar
>>> calendar.month_name[1:]
['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December']
```

O la de los días de la semana con:

```
>>> import calendar
>>> calendar.day_name[:]
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
```

También incluye constantes de los días de la semana y algunos meses (no sé por qué on están todos...) que corresponde con su número, por ejemplo `calendar.February` vale 2 o `calendar.TUESDAY` vale 1 y `calendar.MONDAY` vale 0

Usando esto podemos recuperar el nombre del día de la semana:

```
y para saber el nombre del día usamos calendar
>>> import calendar
>>> from datetime import date
>>> hoy = date.today()
>>> print(calendar.day_name[hoy.weekday()])
monday
```

Para obtener los nombres de los meses y días en el idioma local debemos establecer la localización con el módulo *locale*. Por ejemplo para establecer el “español” de “España” haríamos

```
>>> import locale
>>> locale.setlocale(locale.LC_ALL, 'es_ES.UTF-8')
'es_ES.UTF-8'
```

A partir de este momento este intérprete mostrará los resultados traducidos:

```
>>> import calendar
>>> calendar.month_name[1:]
['enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio', 'agosto',
'septiembre', 'octubre', 'noviembre', 'diciembre']
>>> calendar.day_name[:]
['lunes', 'martes', 'miércoles', 'jueves', 'viernes', 'sábado', 'domingo']
```

## Recuperando fechas de texto

A veces queremos recuperar una fecha de un texto, es decir, tenemos una cadena que sabemos que incluye información de fecha y de hora y queremos recuperar un *datetime* para poder utilizarlo de una forma más sencilla.

Para ello utilizaremos el método **parse** de la clase *datetime.parser* que nos permite recuperar tanto la fecha desde una cadena que sabemos que contiene una fecha, como desde un texto, por ejemplo una línea de un mensaje de error de un programa. A partir de ahí también podemos recuperar una fecha concreta.

Si tenemos seguridad de que la cadena incluye sólo una fecha lo usaremos así:

```
>>> from dateutil.parser import parse
>>> fechaStr = '10-5-2021 18:45:12'
>>> parse(fechaStr)
datetime.datetime(2021, 10, 5, 18, 45, 12)
```

En el caso de la fecha esté incluida dentro de un texto podemos usar el parámetro *fuzzy=True* que buscará la parte de la fecha y nos la devolverá:

```
>>> from dateutil.parser import parse
>>> log_line = 'INFO 2014-07-03T23:27:51 supybot Shutdown complete.'
```

```
>>> parse(log_line, fuzzy=True)
datetime.datetime(2014, 7, 3, 23, 27, 51)
```

También podemos instalar otro paquete que nos permite parsear fechas, como es **dateparser**:

```
pip3 install dateparser
```

Usando su método `parse` podemos convertir un texto a fecha, obteniendo un *datetime*. También funciona con fechas escritas en diferentes idiomas:

```
>>> from dateparser import parse
>>> fecha = '18 de mayo de 2004'
>>> parse(fecha)
datetime.datetime(2004, 5, 18, 0, 0)
```

## Instalación de nuevo módulos

Si bien es cierto que son muchos los módulos que incluye la librería estándar, en el mundo Python tenemos la suerte de que miles de programadores están constantemente desarrollando, mejorando y creando nuevos módulos que a veces sólo facilitan algunas tareas mientras que otras abren nuevos campos de aplicación.

Por eso es importante tener un sistema de administración y gestión de módulos para poder instalar y actualizar los paquetes instalados.

Por defecto en todas las instalaciones de Python, a partir de la versión 3.4, se incluye la herramienta *pip* (*Pip installs packages*, ¡sí el nombre es recursivo!) que nos facilita la tarea de instalar paquetes que no forman parte de la librería estándar.

Y ¿dónde podemos encontrar estos módulos? El mejor repositorio de paquetes es [Python Package Index](https://pypi.org/) (PyPI)



PyPi Logo

El proceso de instalación de un paquete con pip tiene varias etapas:

- Se localiza el paquete en el repositorio de pip.
- Se estudian sus dependencias (los paquetes de los que depende y las versiones necesarias para que funcione).
- Se comprueba si los paquetes de los que depende están instalados, y en las versiones correspondientes.
- Se instalan aquellos que nos faltan.
- Si se cumplen todas las dependencias se instala el paquete
- La instalación de los paquetes hace que estos estén disponibles desde nuestro código

A veces para la instalación de un paquete python es necesario que tengamos instalada una librería o un software previamente. Algunos paquetes están configurados para descargar el código fuente de estas librerías y compilarlo si no las tenemos instaladas previamente.

## Uso de pip

Si queremos instalar un módulo con pip haremos desde la consola de nuestro sistema operativo (no desde dentro del intérprete Python)

```
$ pip install paquete
```

La herramienta *pip* buscará en el repositorio *PyPi* ese paquete, detectará sus dependencias e instalará todos los módulos necesarios para que “paquete” funcione.

Si usamos Google Colab usaremos el comando `!pip` para las instalaciones, y dado que los ficheros se borrarán al desconectar, tendremos que incluir unas instrucciones al principio de nuestro documento para que se instale todo lo necesario antes de que podamos utilizarlo.

### Install the required libs

```
[2] #@title Install the required libs
!pip install -qq git+https://github.com/huggingface/diffusers.git
!pip install -qq "ipywidgets>=7,<8"
!pip install -qq bitsandbytes
```

```
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing wheel metadata ... done
```

Cuando instalamos en Linux con pip podemos hacerlo como usuario o como administrador, el resultado es que se instalará en nuestro directorio personal o en el

sistema. Si instalas como administrador corremos el riesgo de alterar el funcionamiento del sistema.

Al usar pip, se comprueba si existe una nueva versión disponible y en ese caso aparece un mensaje que nos recomienda actualizarlo con el comando

```
$ python3 -m pip install --upgrade pip
```

Si por alguna razón tenemos instalado Python 2 en nuestro sistema podemos asegurarnos de que se instala el paquete para la versión 3 usando *pip3* en lugar de pip.

Vamos a ver más opciones de pip:

- Podemos saber la versión de *pip* que tenemos instalada con:  

```
$ pip --version
```
- Como hemos dicho antes, para instalar un paquete haremos:  

```
$ pip install PackageName
```
- Podemos elegir que se instale una versión concreta de un paquete con:  

```
$ pip install 'PackageName==1.4'
```
- Para actualizar un paquete haremos:  

```
$ pip install --upgrade PackageName
```
- Podemos desinstalar un paquete con:  

```
$ pip uninstall PackageName
```
- Para mostrar los módulos que tenemos instalados usaremos:  

```
$ pip list
```
- Podemos ver los detalles de un módulo que tenemos instalado con:  

```
$ pip show requests
```

Entre los detalles aparecen los módulos que se requieren para usar el módulo y los módulos que requieren de éste.

Hagamos un ejemplo. Voy a instalar el módulo *requests* que usaremos más adelante y que nos permite acceder desde nuestro código a páginas webs:

```
$ pip3 install requests
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: requests in
/home/javacasm/.local/lib/python3.6/site-packages (2.22.0)
Collecting idna<2.9,>=2.5
 Using cached idna-2.8-py2.py3-none-any.whl (58 kB)
```

```

Requirement already satisfied: chardet<3.1.0,>=3.0.2 in
/home/javacasm/.local/lib/python3.6/site-packages (from requests) (3.0.4)
Requirement already satisfied: certifi>=2017.4.17 in
/home/javacasm/.local/lib/python3.6/site-packages (from requests)
(2019.11.28)
Requirement already satisfied: urllib3!=1.25.0,!<1.25.1,<1.26,>=1.21.1 in
/home/javacasm/.local/lib/python3.6/site-packages (from requests) (1.25.8)
Installing collected packages: idna
 Attempting uninstall: idna
 Found existing installation: idna 2.9
 Uninstalling idna-2.9:
 Successfully uninstalled idna-2.9
Successfully installed idna-2.8

```

- En la primera línea nos dice que al ser un usuario normal no puede instalar en el sistema y se hará la instalación en el directorio del usuario.
- Vemos las dependencias y cómo algunas de ellas ya están instaladas.
- En alguna de las dependencias se muestra que la versión debe cumplir algunas condiciones: urllib3!=1.25.0 ó chardet<3.1.0,>=3.0.2
- Podemos generar un listado de todos los módulos que necesitamos para ejecutar algo y guardarlo en un fichero:

```
$ pip freeze > requisitos.txt
```

En el fichero quedarán los paquetes instalados y las versiones instaladas.

Si queremos reproducir esa configuración de instalación lo haremos con:

```
$ pip install -r requisitos.txt
```

Esto es algo que se hace en entornos profesionales para reproducir la configuración de una máquina en otra.